

第07章：LangChain使用之Retrieval

讲师：尚硅谷-宋红康

官网：[尚硅谷](#)

Retrieval直接翻译过来即“检索”，本章Retrieval模块包括与检索步骤相关的所有内容，例如数据的获取、切分、向量化、向量存储、向量检索等模块。常被应用于构建一个“企业/私人的知识库”，提升大模型的整体能力。

1、Retrieval模块的设计意义

1.1 大模型的幻觉问题

拥有记忆后，确实扩展了AI工程的应用场景。

但是在专有领域，LLM无法学习到所有的专业知识细节，因此在面向专业领域知识的提问时，无法给出可靠准确的回答，甚至会“胡言乱语”，这种现象称之为LLM的“幻觉”。

大模型生成内容的不可控，尤其是在金融和医疗领域等领域，一次金额评估的错误，一次医疗诊断的失误，哪怕只出现一次都是致命的。但，对于非专业人士来说可能难以辨识。目前还没有能够百分之百解决这种情况的方案。



当前大家普遍达成共识的一个方案：

首先，为大模型提供一定的上下文信息，让其输出会变得更稳定。

其次，利用本章的RAG，将检索出来的文档和提示词输送给大模型，生成更可靠的答案。

1.2 RAG的解决方案

可以说，当应用需求集中在利用大模型去回答特定私有领域的知识，且知识库足够大，那么除了微调大模型外，RAG就是非常有效的一种缓解大模型推理的“幻觉”问题的解决方案。

LangChain对这一流程提供了解决方案。

如果说LangChain相当于给LLM这个“大脑”安装了“四肢和躯干”，RAG则是为LLM提供了接入“人类知识图书馆”的能力。

目前，已经出现了非常多的产品几乎完全建立在 RAG 之上，包括客服系统、基于大模型的数据分析，以及成千上万的数据驱动聊天应用，应用场景五花八门。



1.3 RAG的优缺点

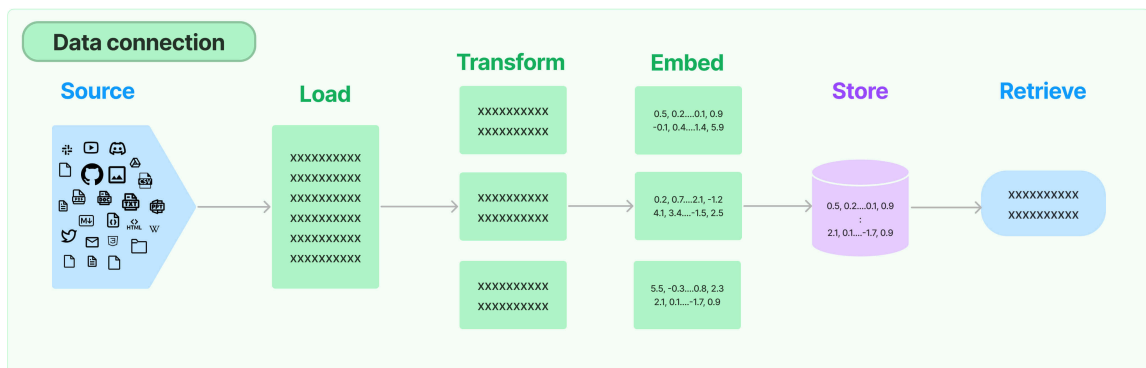
RAG的优点

- 1) 相比提示词工程，RAG有 **更丰富的上下文和数据样本**，可以不需要用户提供过多的背景描述，就能生成比较符合用户预期的答案。
- 2) 相比于模型微调，RAG可以提升问答内容的 **时效性** 和 **可靠性**
- 3) 在一定程度上保护了业务数据的 **隐私性**。

RAG的缺点

- 1) 由于每次问答都涉及外部系统数据检索，因此RAG的 **响应时延** 相对较高。
- 2) 引用的外部知识数据会 **消耗大量的模型Token** 资源。

1.4 Retrieval流程



环节1: Source (数据源)

指的是RAG架构中所外挂的知识库。这里有三点说明：

- 1、原始数据源类型多样：如：视频、图片、文本、代码、文档等
- 2、形式的多样性：
 - 可以是上百个.csv文件，可以是上千个.json文件，也可以是上万个.pdf文件
 - 可以是某一个业务流程外放的API，可以是某个网站的实时数据等

环节2: Load (加载)

文档加载器 (Document Loaders) 负责将来自不同数据源的非结构化文本，加载到 **内存**，成为 **文档 (Document)对象**。

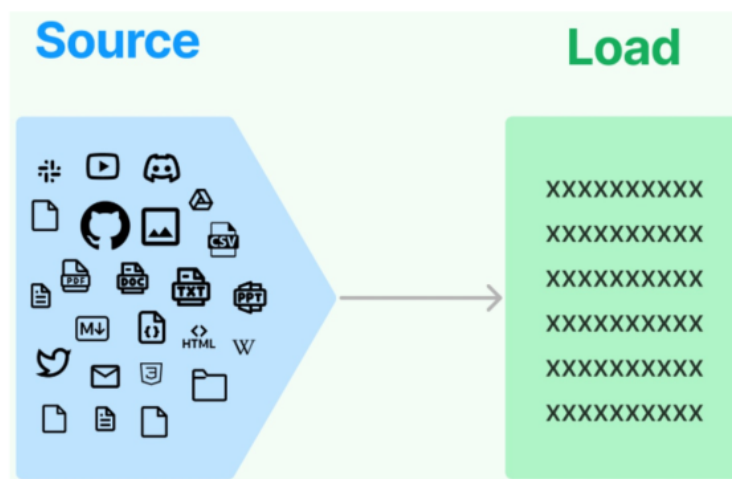
文档对象包含 **文档内容** 和相关 **元数据信息**，例如TXT、CSV、HTML、JSON、Markdown、PDF，甚至YouTube 视频转录等。

典型数据格式：

- CSV
- HTML
- JSON
- Markdown
- PDF
- File Directory
- ...

部分典型数据源：

- ArXiv
- BiliBili
- Discord
- Figma
- GitHub
- Reddit
- TensorFlow Datasets
- ...



文档加载器还支持 “**延迟加载**” 模式，以缓解处理大文件时的内存压力。

文档加载器的编程接口使用起来非常简单，以下给出加载TXT格式文档的例子。

```
1 from langchain.document_loaders import TextLoader
2
3 text_loader = TextLoader("./test.txt")
4
5 docs = text_loader.load() #返回List列表(Document对象)
6 print(docs)
```

环节3: Transform (转换)

文档转换器 (Document Transformers) 负责对加载的文档进行转换和处理，以便更好地适应下游任务的需求。

文档转换器提供了一致的接口（工具）来操作文档，主要包括以下几类：

- **文本拆分器 (Text Splitters)**：将长文本拆分成语义上相关的小块，以适应语言模型的上下文窗口限制。
- **冗余过滤器 (Redundancy Filters)**：识别并过滤重复的文档。

- **元数据提取器(Metadata Extractors)**：从文档中提取标题、语调等结构化元数据。
- **多语言转换器(Multi-lingual Transformers)**：实现文档的机器翻译。
- **对话转换器(Conversational Transformers)**：将非结构化对话转换为问答格式的文档。

总的来说，文档转换器是 LangChain 处理管道中非常重要的一个组件，它丰富了框架对文档的表示和操作能力。

在这些功能中，文档拆分器是必须的操作。下面单独说明。

环节3.1: Text Splitting (文档拆分)

- **拆分/分块的必要性**：前一个环节加载后的文档对象可以直接传入文档拆分器进行拆分，而文档切块后才能 **向量化** 并存入数据库中。
- **文档拆分器的多样性**：LangChain提供了丰富的文档拆分器，不仅能够切分普通文本，还能切分 Markdown、JSON、HTML、代码等特殊格式的文本。
- **拆分/分块的挑战性**：实际拆分操作中需要处理许多细节问题，**不同类型的文本**、**不同的使用场景** 都需要采用不同的分块策略。
 - 可以按照 **数据类型** 进行切片处理，比如针对 **文本类数据**，可以直接按照字符、段落进行切片；**代码类数据** 则需要进一步细分以保证代码的功能性；
 - 可以直接根据 **token** 进行切片处理

在构建RAG应用程序的整个流程中，**拆分/分块是最具挑战性的环节之一，它显著影响检索效果**。目前还没有通用的方法可以明确指出哪一种分块策略最为有效。不同的使用场景和数据类型都会影响分块策略的选择。

环节4: Embed (嵌入)

文档嵌入模型 (Text Embedding Models) 负责将 **文本** 转换为 **向量表示**，即**模型赋予了文本计算机可理解的数值表示**，使文本可用于向量空间中的各种运算，大大拓展了文本分析的可能性，是自然语言处理领域非常重要的技术。

举例：

1. The:

- 假设词嵌入为： $\text{Embedding}_{\text{The}} = [0.9, 2.1, 4.3]$

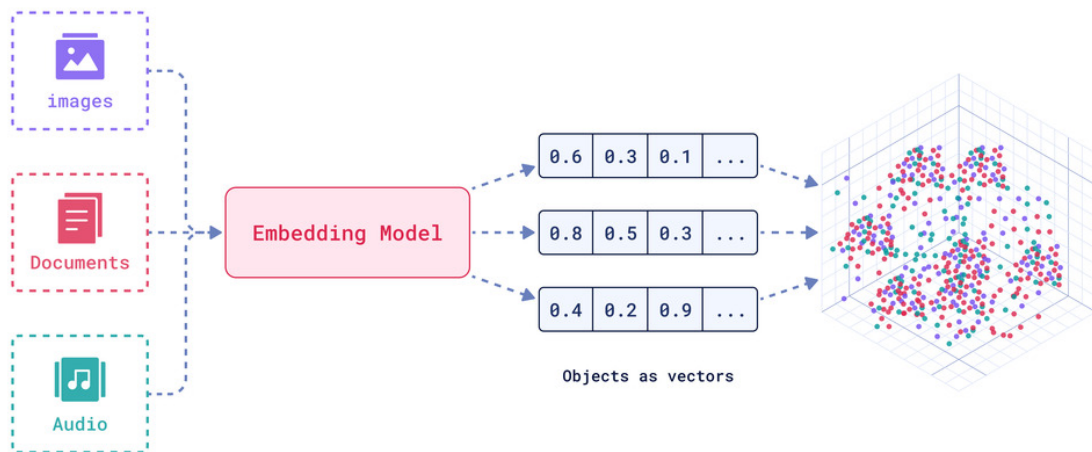
2. cat:

- 之前给出的嵌入是： $\text{Embedding}_{\text{cat}} = [1.2, 3.4, 5.6]$

3. sat:

- 假设词嵌入为： $\text{Embedding}_{\text{sat}} = [2.5, 3.6, 4.7]$

- 实现原理：通过 **特定算法**（如Word2Vec）将语义信息编码为固定维度的向量，具体算法细节需后续深入。
- 关键特性：**相似的词在向量空间中距离相近**，例如"猫"和"犬"的向量夹角小于"猫"和"汽车"。

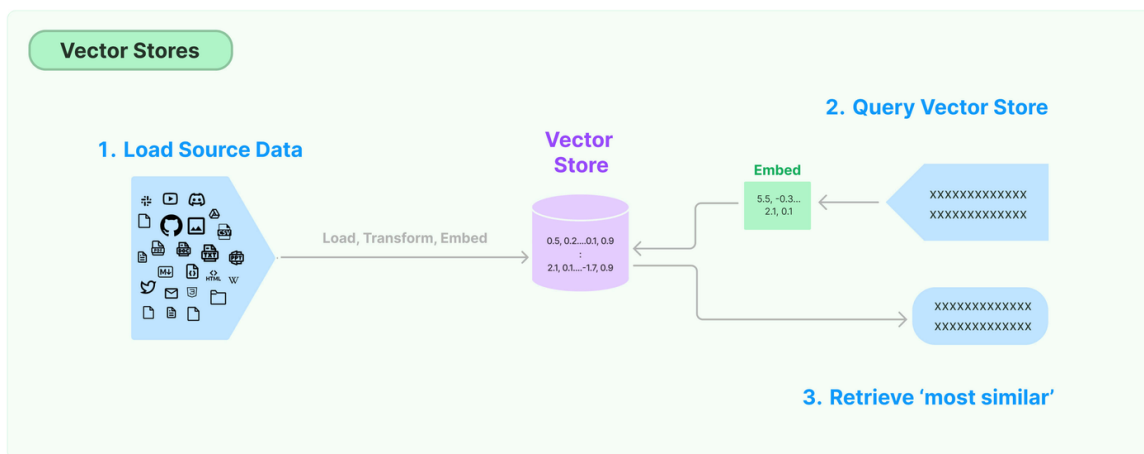


文本嵌入为 LangChain 中的问答、检索、推荐等功能提供了重要支持。具体为：

- **语义匹配**：通过计算两个文本的向量余弦相似度，判断它们在语义上的相似程度，实现语义匹配。
- **文本检索**：通过计算不同文本之间的向量相似度，可以实现语义搜索，找到向量空间中最相似的文本。
- **信息推荐**：根据用户的历史记录或兴趣嵌入生成用户向量，计算不同信息的向量与用户向量的相似度，推荐相似的信息。
- **知识挖掘**：可以通过聚类、降维等手段分析文本向量的分布，发现文本之间的潜在关联，挖掘知识。
- **自然语言处理**：将词语、句子等表示为稠密向量，为神经网络等下游任务提供输入。

环节5: Store (存储)

LangChain 还支持把文本嵌入存储到向量存储或临时缓存，以避免需要重新计算它们。这里就出现了数据库，支持这些嵌入的高效 **存储** 和 **搜索** 的需求。



环节6: Retrieve (检索)

检索器 (Retrievers) 是一种用于 **响应非结构化查询** 的接口，它可以返回符合查询要求的文档。

LangChain 提供了一些常用的检索器，如 **向量检索器**、**文档检索器**、**网站研究检索器** 等。

通过配置不同的检索器，LangChain 可以灵活地平衡检索的精度、召回率与效率。检索结果将为后续的问答生成提供信息支持，以产生更加准确和完整的回答。

2、文档加载器 Document Loaders

LangChain的设计：对于 **Source** 中多种不同的数据源，我们可以用一种统一的形式读取、调用。

2.1 加载txt

```
1  # 1.导入相关依赖
2  from langchain.document_loaders import TextLoader
3
4  # 2.定义TextLoader对象, file_path=".txt的位置"
5  text_loader = TextLoader(file_path="asset/load/01-langchain-utf-8.txt", encoding="utf-8")
6
7  # 3.加载
8  docs = text_loader.load() #返回List列表(Document对象)
9
10 # 4.打印
11 print(docs)
```

```
[Document(metadata={'source': 'asset/load/01-langchain-utf-8.txt'},
page_content='LangChain 是一个用于构建基于大语言模型（LLM）应用的开发框架，旨在帮助
开发者更高效地集成、管理和增强大语言模型的能力，构建端到端的应用程序。它提供了一套模块
化工具和接口，支持从简单的文本生成到复杂的多步骤推理任务']
```

Document对象中有两个重要的属性：

- page_content：真正的文档内容
- metadata：文档内容的原数据

```
1  type(docs[0]) #langchain_core.documents.base.Document
```

```
1  docs[0].page_content
2  '''
3  LangChain 是一个用于开发由大型语言模型 (LLMs) 驱动的应用程序的框架。LangChain简化了LLM应用
程序生命周期的每个阶段。 \nLangChain 已经成为了我们每一个大模型开发工程师的标配。
4  '''
```

```
1  docs[0].metadata # {'source': './data/langchain.txt'}
```

2.2 加载pdf

举例1：

LangChain加载PDF文件使用的是pypdf，先安装

```
1  pip install pypdf
```

```
1  # 1.导入相关的依赖 PyPDFLoader()
```



```

2 from langchain.document_loaders import PyPDFLoader
3
4 # 2.定义PyPDFLoader
5 pdfLoader = PyPDFLoader(file_path= "asset/load/02-load.pdf")
6
7 # 3.加载
8 docs = pdfLoader.load()
9
10 print(docs)
11 print(type(docs[0]))
12
13 ## 4.遍历集合
14 # for doc in docs:
15 #     print(f"加载的文档:{doc.page_content}")
16 #

```

```

[Document(metadata={'producer': 'Microsoft® Word 2019', 'creator': 'Microsoft® Word 2019', 'creationdate': '2025-06-20T17:18:19+08:00', 'moddate': '2025-06-20T17:18:19+08:00', 'source': 'asset/load/02-load.pdf', 'total_pages': 1, 'page': 0, 'page_label': '1'}, page_content="他的车，他的命！他忽然想起来，一年，二年，至少有三四年；一滴汗，两滴汗，不\n知道多少万滴汗，才挣出那辆车。从风里雨里的咬牙，从饭里茶里的自苦，才赚出那辆车。那辆车是他的一切挣扎与困苦的总结果与报酬，像身经百战的武士的一颗徽章。……他老想\n着远远的一辆车，可以使他自由，独立，像自己的手脚的那么一辆车。"\n\n"他吃，他喝，他嫖，他赌，他懒，他狡猾，因为他没了心，他的心被人家摘了去。他\n只剩下那个高大的肉架子，等着溃烂，预备着到乱死岗子去。……体面的、要强的、好梦想\n的、利己的、个人的、健壮的、伟大的祥子，不知陪着人家送了多少回殡；不知道何时何地\n会埋起他自己来，埋起这堕落的、自私的、不幸的、社会病胎里的产儿，个人主义的末路鬼！\n")]
<class 'langchain_core.documents.base.Document'>

```

同样的：

```

1 type(pages[0]) #langchain_core.documents.base.Document

```

```

1 pages[0].page_content #只获取本页内容

```

```

1 pages[0].metadata # {..., 'source': './asset/load/load.pdf',..., 'page': 0}

```

举例2：

```

1 #1.导入相关依赖
2 from langchain.document_loaders import PyPDFLoader
3
4 # 2.定义PyPDFLoader对象,加载在线的pdf文件
5 py_pdfLoader = PyPDFLoader(file_path= "https://arxiv.org/pdf/2302.03803")
6
7 # 3.加载
8 docs = py_pdfLoader.load()
9
10 print(len(docs)) # 8

```

```

11
12 # 4.打印
13 for doc in docs:
14     print(doc)

```

```

8
page_content='arXiv:2302.03803v1 [math.AG] 7 Feb 2023
A WEAK (k, k)-LEFSCHETZ THEOREM FOR PROJECTIVE
TORIC ORBIFOLDS
William D. Montoya
Instituto de Matemática, Estatística e Computação Científica,
Universidade Estadual de Campinas (UNICAMP),
Rua Sérgio Buarque de Holanda 651, 13083-859, Campinas, SP, Brazil
February 9, 2023
Abstract
Firstly we show a generalization of the (1,1)-Lefschetz theorem for projective
toric orbifolds and secondly we prove that on 2k-dimensional quasi-smooth hyper-
surfaces coming from quasi-smooth intersection surfaces, under the Cayley trick,
every rational (k, k)-cohomology class is algebraic, i.e., the Hodge conjecture holds
on them.
1 Introduction
In [3] we proved that, under suitable conditions, on a very general codimension s quasi-
smooth intersection subvariety X in a projective toric orbifold Pd
Σ with d + s = 2(k + 1)
the Hodge conjecture holds, that is, every (p, p)-cohomology class, under the Poincaré
duality is a rational linear combination of fundamental classes of algebraic subvarieties
.... 略

```

举例3：使用load_and_split()

```

1 # 1.导入相关的依赖 PyPDFLoader()
2 from langchain.document_loaders import PyPDFLoader
3
4 # 2.定义PyPDFLoader
5 py_pdfLoader = PyPDFLoader(file_path= "./asset/load/load.pdf")
6
7 # 3.加载
8 docs = py_pdfLoader.load_and_split() #底层默认使用了递归字符文本切分器
9
10 print(docs)

```

同样，对于 **PyPDFLoader**，依然是使用 **.page_content** 和 **.metadata** 去访问数据，也就是说，每一个文档加载器虽然代码逻辑不同，应用需求不同，但使用方式是相同的。

2.3 加载CSV

举例1：加载csv所有列

```
1 from langchain_community.document_loaders.csv_loader import CSVLoader
2
3 loader = CSVLoader(file_path="asset/load/03-load.csv")
4 data = loader.load()
5 print(data)
6
7 print(type(data)) # <class 'list'>
8 print(type(data[0])) # <class 'langchain_core.documents.base.Document'>
9 print(len(data)) # 4
10 print(data[0].page_content) # id: 1 title: Introduction to Python ...
```

```
[Document(metadata={'source': 'asset/load/04-load.csv', 'row': 0}, page_content='id:
1\ntitle: Introduction to Python\ncontent: Python is a popular programming
language.\nauthor: John Doe'), Document(metadata={'source': 'asset/load/04-load.csv',
'row': 1}, page_content='id: 2\ntitle: Data Science Basics\ncontent: Data science involves
statistics and machine learning.\nauthor: Jane Smith'), Document(metadata={'source':
'asset/load/04-load.csv', 'row': 2}, page_content='id: 3\ntitle: Web Development\ncontent:
HTML, CSS and JavaScript are core web technologies.\nauthor: Mike Johnson'),
Document(metadata={'source': 'asset/load/04-load.csv', 'row': 3}, page_content='id:
4\ntitle: Artificial Intelligence\ncontent: AI is transforming many industries.\nauthor:
Sarah Williams')]
<class 'list'>
<class 'langchain_core.documents.base.Document'>
4
id: 1
title: Introduction to Python
content: Python is a popular programming language.
author: John Doe
```

举例2：

使用 `source_column` 参数指定文件加载的列，保存在`source`变量中。

```
1 from langchain_community.document_loaders.csv_loader import CSVLoader
2
3 loader = CSVLoader(
4     file_path="./asset/load/03-load.csv",
5     source_column='author'
6 )
7
8 data = loader.load()
9 print(data)
```

```
[Document(metadata={'source': 'John Doe', 'row': 0}, page_content='id: 1\ntitle:
Introduction to Python\ncontent: Python is a popular programming language.\nauthor:
John Doe'), Document(metadata={'source': 'Jane Smith', 'row': 1}, page_content='id:
2\ntitle: Data Science Basics\ncontent: Data science involves statistics and machine
```

```
learning.\nauthor: Jane Smith'), Document(metadata={'source': 'Mike Johnson', 'row': 2},
page_content='id: 3\nitle: Web Development\ncontent: HTML, CSS and JavaScript are
core web technologies.\nauthor: Mike Johnson'), Document(metadata={'source': 'Sarah
Williams', 'row': 3}, page_content='id: 4\nitle: Artificial Intelligence\ncontent: AI is
transforming many industries.\nauthor: Sarah Williams'])
```

2.4 加载JSON

LangChain提供的JSON格式的文档加载器是 **JSONLoader**。在实际应用场景中，JSON格式的数据占有很大比例，而且JSON的形式也是多样的。我们需要特别关注。

JSONLoader 使用指定的 jq 结构来解析 JSON 文件。jq 是一个轻量级的命令行 JSON 处理器，可以对 JSON 格式的数据进行各种复杂的处理，包括数据过滤、映射、减少和转换，是处理 JSON 数据的首选工具之一。

```
1 pip install jq
```

举例1：使用JSONLoader文档加载器加载

```
1  # 1.导入依赖
2  from langchain_community.document_loaders import JSONLoader
3  from pprint import pprint
4
5  # 2.定义JSONLoader对象
6  # 错误的
7  # json_loader=JSONLoader(file_path="asset/load/04-load.json")
8
9  # 情况1
10 # json_loader=JSONLoader(
11 #     file_path="asset/load/04-load.json",
12 #     jq_schema=".", #直接提取完整的JSON对象（包括所有字段）
13 #     text_content=False #保持原始JSON结构，将提取的数据转换为JSON字符串存入page_content
    字段中
14 # )
15
16 # 情况2
17 # .messages[].content:遍历.messages[]中所有元素 从每一个元素中提取.content字段
18 json_loader=JSONLoader(
19     file_path="asset/load/04-load.json",
20     jq_schema=".messages[].content"
21 )
22
23 # 3.加载
24 docs = json_loader.load()
25 pprint(docs)
26
27 # 4.提取content中指定字符数的内容
28 # print(docs[0].page_content[:10])
```

```
[Document(metadata={'source':
'D:\develop\workspace\vscode\langchain\asset\load\load.json', 'seq_num': 1},
page_content='Hello, how are you today?'),
Document(metadata={'source':
'D:\develop\workspace\vscode\langchain\asset\load\load.json', 'seq_num': 2},
page_content='I'm doing well, thanks for asking!'),
Document(metadata={'source':
'D:\develop\workspace\vscode\langchain\asset\load\load.json', 'seq_num': 3},
page_content='Would you like to meet for lunch?'),
Document(metadata={'source':
'D:\develop\workspace\vscode\langchain\asset\load\load.json', 'seq_num': 4},
page_content='Sure, that sounds great!')]
```

Hello, how

举例2：提取04-response.json文件中嵌套在 data.items[].content 的文本

- 如果希望处理 JSON 中的 **嵌套字段**、**数组元素提取**，可以使用 `content_key` 配合 `is_content_key_jq_parsable=True`，通过 jq 语法精准定位目标数据。
- 通常，对api请求结果的采集

```
1  # 1.导入相关依赖
2  from langchain_community.document_loaders import JSONLoader
3  from pprint import pprint
4
5  # 2.定义json文件的路径
6  file_path = 'asset/load/04-response.json'
7
8  # 3.定义JSONLoader对象
9  # 提取嵌套在 data.items[].content 的文本，并保留其他字段作为元数据
10 # 方式1:
11 # loader = JSONLoader(
12 #     file_path=file_path,
13 #     jq_schema=".data.items[].content",
14 # )
15
16
17 # 方式2:
18 loader = JSONLoader(
19     file_path=file_path,
20     jq_schema=".data.items[]", # 先定位到数组条目
21     content_key=".content", # 再从条目中提取 content 字段
22     is_content_key_jq_parsable=True # 用jq解析content_key
23 )
24
25
26 # 4.加载
27 data = loader.load()
28 pprint(data)
29
30 pprint(data[0].page_content)
```

```
[Document(metadata={'source':
'D:\code\workspace_pycharm_test\langchain_test\chapter_new7_RAG\asset\load\04-
response.json', 'seq_num': 1}, page_content='This article explains how to parse API
responses...'),
Document(metadata={'source':
'D:\code\workspace_pycharm_test\langchain_test\chapter_new7_RAG\asset\load\04-
response.json', 'seq_num': 2}, page_content='Learn to handle nested structures with...'),
Document(metadata={'source':
'D:\code\workspace_pycharm_test\langchain_test\chapter_new7_RAG\asset\load\04-
response.json', 'seq_num': 3}, page_content='Best practices for preserving metadata...')]

'This article explains how to parse API responses...'
```

举例3：提取04-response.json文件中嵌套在 data.items[] 里的 title、content 和 其文本

```
1  # 1.导入相关依赖
2  from langchain_community.document_loaders import JSONLoader
3  from pprint import pprint
4
5  # 2.定义json文件的路径
6  file_path = 'asset/load/04-response.json'
7
8  # 3.定义JSONLoader对象
9  # 提取嵌套在 data.items[].content 的文本，并保留其他字段作为元数据
10 # loader = JSONLoader(
11 #     file_path=file_path,
12 #     jq_schema=".data.items[] | {id, author, text: (.title + '\n' + .content)}",
13 #     jq_schema="".data.items[] | {
14 #         id,
15 #         author,
16 #         created_at,
17 #         title, # 保留title字段
18 #         text: (.title + "\n" + .content)
19 #     },
20 #     content_key=".text", # 再从条目中提取 content 字段
21 #     is_content_key_jq_parsable=True # 用jq解析content_key
22 # )
23 loader = JSONLoader(
24     file_path=file_path,
25     jq_schema=".data.items[] | {id, author, text: (.title + '\n' + .content)}",
26     jq_schema=".data.items[]",
27     content_key='.title + "\n\n" + .content',
28     is_content_key_jq_parsable=True # 用jq解析content_key
29 )
30
31 # loader = JSONLoader(
32 #     file_path=file_path,
33 #     jq_schema=".data.items[] | {id, author, text: (.title + '\n' + .content)}",
34 #     jq_schema=""
35 #     .data.items[] | {
36 #         metadata: {
37 #             id,
```

```

38 #         author,
39 #         created_at
40 #     },
41 #     content: (.title + "\n\n" + .content)
42 # }
43 # ''' # 构建新结构
44 # content_key='.title + "\n\n" + .content',
45 # is_content_key_jq_parsable=True # 用jq解析content_key
46 # )
47
48 # 4.加载
49 data = loader.load()
50 pprint(data)
51
52 pprint(data[0].page_content)

```

2.5 加载HTML(了解)

```
1 pip install unstructured
```

举例：

```

1 # 1.导入相关的依赖
2 from langchain.document_loaders import UnstructuredHTMLLoader
3
4 # 2.定义UnstructuredHTMLLoader对象
5 # strategy:
6 # "fast" 解析加载html文件速度是比较快 (但可能丢失部分结构或元数据)
7 # "hi_res": (高分辨率解析) 解析精准 (速度慢一些)
8 # "ocr_only" 强制使用ocr提取文本, 仅仅适用于图像 (对HTML无效)
9
10 # mode : one of {'paged', 'elements', 'single'}
11 # "elements" 按语义元素 (标题、段落、列表、表格等) 拆分成多个独立的小文档
12
13 html_loader = UnstructuredHTMLLoader(
14     file_path="asset/load/05-load.html",
15     mode="elements",
16     strategy="fast"
17 )
18
19 # 3.加载
20 docs = html_loader.load()
21
22 print(len(docs)) # 16
23
24 # 4.打印
25 for doc in docs:
26     print(doc)
27

```

page_content='首发于自然语言处理算法与实践' metadata={'source': 'asset/load/05-load.html', 'last_modified': '2025-07-17T15:38:36', 'languages': ['zho'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'category': 'UncategorizedText', 'element_id': 'b082a3e1f4714ffa5f25741f39d82c17'}

page_content='RAG:将检索与生成方式相结合来做生成任务' metadata={'source': 'asset/load/05-load.html', 'category_depth': 0, 'last_modified': '2025-07-17T15:38:36', 'languages': ['kor'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'category': 'Title', 'element_id': '46103fd31eae47ed36481d13185af8a9'}

page_content='烛之文' metadata={'source': 'asset/load/05-load.html', 'last_modified': '2025-07-17T15:38:36', 'languages': ['kor'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '46103fd31eae47ed36481d13185af8a9', 'category': 'UncategorizedText', 'element_id': 'e02798c2e2bb964165a9e9356b82a3f6'}

page_content='1、前言' metadata={'source': 'asset/load/05-load.html', 'category_depth': 1, 'last_modified': '2025-07-17T15:38:36', 'languages': ['zho'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '46103fd31eae47ed36481d13185af8a9', 'category': 'Title', 'element_id': '683a24e897e3a9b862ead6c7979a58dc'}

page_content='在上一篇<kNN-NER: 利用knn近邻算法来做命名实体识别>提及到文中提出kNN-NER框架是一种检索式增强的方法（retrieval augmented methods），就去查看有关retrieval augmented的paper，了解其核心思想，觉得检索式增强的方法很适合许多业务场景使用，因其以一种简捷的方式将外部知识融于模型中去。今天就分享一篇来自Facebook AI Research的paper，论文提出一种检索式增强生成方法，应用于知识密集型的NLP任务（如问答生成），该篇论文被2020年NeurIPS会议接收。' metadata={'source': 'asset/load/05-load.html', 'last_modified': '2025-07-17T15:38:36', 'languages': ['nor'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '683a24e897e3a9b862ead6c7979a58dc', 'category': 'NarrativeText', 'element_id': 'd345b4a58c84984eb1acf1105fd9f214'}

page_content='文中说到，以BERT之类的大规模预训练模型将很多事实知识信息存入模型中，可以看着是pre-trained parametric类型，尽管以fine-tuned方式在下游任务取得显著的成效，但这类方法仍存在无法精准地获取和操作知识的缺陷。而在上述提及的问题上，传统知识检索的方法能很好的应对，这类方法可以看着是non-parametric memory类型。于是，论文提出检索式增强生成方法（retrieval-augmented generation, RAG），主要思想就是将pre-trained parametric与non-parametric memory结合起来做语言生成任务，将两类模型集成起来提高任务处理效果。' metadata={'source': 'asset/load/05-load.html', 'last_modified': '2025-07-17T15:38:36', 'languages': ['nor'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '683a24e897e3a9b862ead6c7979a58dc', 'category': 'UncategorizedText', 'element_id': '2fe8d146b5803ec72e5173bf15599710'}

page_content='2、RAG方法' metadata={'source': 'asset/load/05-load.html', 'category_depth': 1, 'last_modified': '2025-07-17T15:38:36', 'languages': ['zho'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '46103fd31eae47ed36481d13185af8a9', 'category': 'Title', 'element_id': '22ca96c9bf71395b8fbbf0928bd7f292'}

page_content='上图为论文提出RAG模型的整体示意图。主要包括两大模块：一个检索器（Retriever, $p(\eta(z/x))$ ）+ 一个生成器（Generator, $p(\theta(y_i|x,z,y_{1:i-1}))$ ）。前者包括query encoder和document index，分别负责query的编码和文档的索引；后者是一个seq2seq的生成模型。在检索中，使用的是最大内积搜索的方法（MIPS）来检索top-K相关文档。' metadata={'source': 'asset/load/05-load.html', 'last_modified': '2025-07-17T15:38:36', 'languages': ['cat', 'nor'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '22ca96c9bf71395b8fbbf0928bd7f292', 'category': 'NarrativeText', 'element_id': '6bbc63aaa7b3afb8d2685e9b3de78a4c'}


```
page_content='3、实验' metadata={'source': 'asset/load/05-load.html', 'category_depth': 1, 'last_modified': '2025-07-17T15:38:36', 'languages': ['zho'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '46103fd31eae47ed36481d13185af8a9', 'category': 'Title', 'element_id': '4df308cd6991fb9e3f0592371bae26be'}
page_content='论文在四类Knowledge-Intensive 任务上进行实验，具体包括开放问答（Open-domain Question Answering）、摘要式问答（Abstractive Question Answering）、开放问题生成（Jeopardy Question Generation）、事实判断（Fact Verification），并使用维基百科（包含2100万个文档）作为检索库。' metadata={'source': 'asset/load/05-load.html', 'last_modified': '2025-07-17T15:38:36', 'languages': ['eng'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '4df308cd6991fb9e3f0592371bae26be', 'category': 'UncategorizedText', 'element_id': 'ed6e043c7fd99ec0824b91725c66e0ba'}
page_content='4、结语' metadata={'source': 'asset/load/05-load.html', 'category_depth': 1, 'last_modified': '2025-07-17T15:38:36', 'languages': ['zho'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '46103fd31eae47ed36481d13185af8a9', 'category': 'Title', 'element_id': '95bc5bffaa5cfd41f5242f9f8b330761'}
page_content='本次分享基于检索增强方式将外部知识融于生成任务中一个新的框架——RAG。对比T5 和 BART这类擅长处理生成任务的模型来说，RAG更新外部知识是不需要重新预训练，成本低；而对比pipeline方法，RAG利用外部知识并不需要构造负责的特征工程。总的来说，RAG方法可作为外部知识融合框架的一种有效实例。' metadata={'source': 'asset/load/05-load.html', 'last_modified': '2025-07-17T15:38:36', 'languages': ['zho', 'kor'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '95bc5bffaa5cfd41f5242f9f8b330761', 'category': 'UncategorizedText', 'element_id': '232dc4ae399e0a8847bfcdeb2e64e215'}
page_content='有兴趣可关注笔者公众号：自然语言处理算法与实践' metadata={'source': 'asset/load/05-load.html', 'last_modified': '2025-07-17T15:38:36', 'languages': ['zho', 'kor'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '95bc5bffaa5cfd41f5242f9f8b330761', 'category': 'UncategorizedText', 'element_id': '41f28a1034fdc4291daf652054c20bd2'}
page_content='编辑于 2022-04-06 10:47' metadata={'source': 'asset/load/05-load.html', 'last_modified': '2025-07-17T15:38:36', 'languages': ['zho'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '95bc5bffaa5cfd41f5242f9f8b330761', 'category': 'UncategorizedText', 'element_id': 'f70255f8bf13d39885508fd845c22382'}
page_content='深度学习（Deep Learning）' metadata={'source': 'asset/load/05-load.html', 'last_modified': '2025-07-17T15:38:36', 'languages': ['nld', 'eng'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '95bc5bffaa5cfd41f5242f9f8b330761', 'category': 'UncategorizedText', 'element_id': '1818d3e8e3a4ce395732bba3428a111d'}
page_content='自然语言处理算法与实践' metadata={'source': 'asset/load/05-load.html', 'category_depth': 2, 'last_modified': '2025-07-17T15:38:36', 'languages': ['kor', 'zho'], 'file_directory': 'asset/load', 'filename': '05-load.html', 'filetype': 'text/html', 'parent_id': '95bc5bffaa5cfd41f5242f9f8b330761', 'category': 'Title', 'element_id': '95058f2148219d97462c5c4bfe175502'}
```

2.6 加载Markdown(了解)

举例1：使用MarkdownLoader加载md文件

```
1 pip install markdown
2
3 pip install unstructured
```

```
1  # 1.导入相关的依赖
2  from langchain.document_loaders import UnstructuredMarkdownLoader
3  from pprint import pprint
4
5  # 2.定义UnstructuredMarkdownLoader对象
6  md_loader = UnstructuredMarkdownLoader(
7      file_path="asset/load/06-load.md",
8      strategy="fast"
9  )
10
11 # 3.加载
12 docs = md_loader.load()
13
14 print(len(docs))
15 # 4.打印
16 for doc in docs:
17     pprint(doc)
```

```
1
Document(metadata={'source': 'asset/load/06-load.md'}, page_content='自然语言处理技术文档\n\n本文档用于测试UnstructuredMarkdownLoader的中文处理能力。 \n\n第一章：简介\n\n自然语言处理(NLP)是人工智能的重要分支，主要技术包括： \n\n文本分类\n\n命名实体识别\n\n机器翻译\n\n情感分析\n\n问答系统\n\n第二章：关键技术\n\n2.1 预训练模型\n\nBERT：双向Transformer编码器\n\nGPT：自回归语言模型\n\nT5：文本到文本转换框架\n\n2.2 代码示例\n\n```\npython from transformers import pipeline\n\n创建文本分类管道\n\nclassifier =\npipeline("text-classification", model="bert-base-chinese")\n\nresult = classifier("这家餐厅\n的服务很棒！") print(result)')
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

举例2：精细分割文档，保留结构信息

将Markdown文档按语义元素（标题、段落、列表、表格等）拆分成多个独立的小文档（**Element** 对象），而不是返回单个大文档。通过指定 **mode="elements"** 轻松保持这种分离。

每个分割后的元素会包含元数据。

```
1  # 1.导入相关的依赖
2  from langchain.document_loaders import UnstructuredMarkdownLoader
3  from pprint import pprint
4
5  # 2.定义UnstructuredMarkdownLoader对象
6  md_loader = UnstructuredMarkdownLoader(
7      file_path="./asset/load/06-load.md",
8      mode="elements",
```

```

9     strategy= "fast"
10 )
11
12 # 3.加载
13 docs = md_loader.load()
14
15 print(len(docs))
16 # 4.打印
17 for doc in docs:
18     # pprint(doc)
19     pprint(doc.page_content)

```

```

19
'自然语言处理技术文档'
'本文档用于测试UnstructuredMarkdownLoader的中文处理能力。'
'第一章：简介'
'自然语言处理(NLP)是人工智能的重要分支，主要技术包括：'
'文本分类'
'命名实体识别'
'机器翻译'
'情感分析'
'问答系统'
'第二章：关键技术'
'2.1 预训练模型'
'BERT：双向Transformer编码器'
'GPT：自回归语言模型'
'T5：文本到文本转换框架'
'2.2 代码示例'
```python from transformers import pipeline'
'创建文本分类管道'
'classifier = pipeline("text-classification", model="bert-base-chinese")'
'result = classifier("这家餐厅的服务很棒！ ") print(result)'

```

## 2.7 加载File Directory(了解)

除了上述的单个文件加载，我们也可以批量加载一个文件夹内的所有文件。

```

1 pip install unstructured

```

举例：

```

1 # 1.导入相关的依赖
2 from langchain.document_loaders import DirectoryLoader
3 from langchain.document_loaders import PythonLoader
4 from pprint import pprint
5
6 # 2.定义DirectoryLoader对象,指定要加载的文件夹路径、要加载的文件类型和是否使用多线程
7 directory_loader = DirectoryLoader(
8 path= "./asset/load",

```

```
9 glob = "*.py",
10 use_multithreading=True,
11 show_progress=True,
12 loader_cls=PythonLoader
13)
14
15 # 3.加载
16 docs = directory_loader.load()
17
18 # 4.打印
19 print(len(docs))
20 for doc in docs:
21 pprint(doc)
```

```

1 100%|██████████| 4/4 [00:00<00:00, 498.83it/s]
2 4
3 Document(metadata={'source': 'asset\\load\\07-fun.py'}, page_content='''\n一 函数入门
\n""\n# 1.不使用函数\n# 打印欢迎信息1\nprint("*****")\nprint("*
)\nprint(" 欢迎来到Python世界 *)\nprint("*
 *)\nprint("*****")\n\n# 打印欢迎信息
2\nprint("*****")\nprint("*
)\nprint(" 欢迎
来到Python世界 *)\nprint("*
 *)\nprint("*****")\n\n# 打印欢迎信息
3\nprint("*****")\nprint("*
)\nprint(" 欢迎
来到Python世界 *)\nprint("*
 *)\nprint("*****")\n\n# 2.使用函数\ndef print_welcome():\n
 """打印欢迎信息"""\n print("*****")\n print("*
)\n print(" 欢迎来到Python世界 *)\n print("*
 *)\n print("*****")\n\n# 多次调用函数打印欢迎信息
\nprint_welcome()\nprint_welcome()\nprint_welcome()')
4 Document(metadata={'source': 'asset\\load\\07-fun_param.py'}, page_content='''\n二 函
数参数\n""\n\n# 1. 无参数版本 - 只能计算固定的购物车\ndef
calculate_total_no_params():\n """计算固定购物车总价"""\n prices = [100, 50, 30] # 商品
价格固定写在函数内\n total = 0\n for price in prices:\n total += price\n return
total\n\n# 只能计算一个固定的购物车\nprint(f"购物车总价:
{calculate_total_no_params()}")\n\n# 2.有参数版本 - 可以计算任意购物车\ndef
calculate_total(prices):\n """计算任意购物车总价"""\n total = 0\n for price in prices:\n
 total += price\n return total\n\n# 可以计算任意购物车\ncart1 = [100, 50, 30]\ncart2 =
[200, 80, 45, 60]\ncart3 = [75, 90, 120]\n\nprint("第一个购物车总价:
{calculate_total(cart1)}")\nprint("第二个购物车总价:{calculate_total(cart2)}")\nprint(f"第三个
购物车总价:{calculate_total(cart3)}")\n\n# 3.参数传递\n# 3.1 不可变类型 函数传递不可变对象
\n\ndef changeInt(a):\n print("函数体中未改变前a的内存地址",id(a))\n a = 10 #底层会创
建一个新对象 然后给新对象一个新值\n print("函数体中改变后a的内存地址",id(a))\n\na = 2 #
创建一个对象 然后给这个对象一个值\nchangeInt(a)\nprint(a)\nprint("函数外b的内存地
址",id(a))\n\n\n# 输出结果\n# 函数体中未改变前a的内存地址 140729722661336\n# 函数体
中改变后a的内存地址 140729722661592\n# 2\n# 函数外b的内存地址
140729722661336\n\n\n# 3.2 可变类型 函数传递不可变对象\n\ndef changeList(myList):\n
myList[1] = 50\n print("函数内的值",myList) # [1,50,3]\n print("函数内列表的内
存",id(myList)) # 0111111\n\nmList = [1,2,3] # 底层创建一个对象 地址
0111111\nchangeList(mList)\nprint("函数外的值",mList) # # [1,50,3]\nprint("函数外列表的内
存",id(mList))\n\n# 输出结果\n# 函数内的值 [1, 50, 3]\n# 函数内列表的内存
1380193079680\n# 函数外的值 [1, 50, 3]\n# 函数外列表的内存 1380193079680\n\n')
5 Document(metadata={'source': 'asset\\load\\07-fun_retun.py'}, page_content='''\n四 函数
的返回值\n""\n\n# 1.返回表达式\n# 2.不带表达式的 return 语句, 返回 None. \n# 3.函数中如果
没有 return 语句, 在函数运行结束后也会返回 None. \n# 4.用变量接收返回结果\n# 5.return 语
句可以返回多个值, 多个值会放在一个元组中. \n\ndef f(a, b, c):\n return a, b, c, [a, b,
c]\nprint(f(1, 2, 3)) # (1, 2, 3, [1, 2, 3])\n')
6 Document(metadata={'source': 'asset\\load\\07-param_form.py'}, page_content='''\n三
函数参数形式\n""\n\n# 1.位置参数\n# 2.关键字参数\n# 3.默认参数\n# 4.不定长参数\n# 4.1 带一
个*\ndef printInfo(num,*vartuple):\n print(num)\n
 print(vartuple)\n\nprintInfo(70,60,50)\n\nprint("- * 20)\n# 如果不定长的参数后面还有参数,必
须通过关键字参数传参\ndef printInfo1(num1,*vartuple,num):\n print(num)\n
 print(num1)\n print(vartuple)\n\nprintInfo1(10,20,num = 40)\n\nprint("- * 20)\n# 如果没
有给不定长的参数传参,那么得到的是空元组\nprintInfo1(70,num = 60)\n\n# 4.2 带二个*\ndef
printInfo(num,**vardict):\n print(num)\n print(vardict)\n #
return\n\nprintInfo(10,key1 = 20,key2 = 30)')

```

## 2.8 BaseLoader、Document源码分析

一方面：LangChain在设计时，要保证Source中多种不同的数据源，在接下来的流程中可以用一种统一的形式读取、调用。

另一方面：为什么 **PDFLoader** 和 **TextLoader** 等Document Loader 都使用 **load()** 去加载，且都使用 **.page\_content** 和 **.metadata** 读取数据。

【解答】每一个在LangChain中集成的文档加载器，都要继承自 **BaseLoader(文档加载器)**，BaseLoader提供了一个名为"load"的公开方法，用于从配置的不同 **数据源** 加载数据，全部作为 **Document** 对象。实现逻辑如下所示：

### BaseLoader类分析

BaseLoader类定义了如何从不同的数据源加载文档，每个基于不同数据源实现的loader，都需要继承 **BaseLoader**。Baseloader要求不多，对于任何具体实现的loader，最少都要实现 load方法。

```
1 class BaseLoader(ABC):
2 """文档加载器接口。
3
4 实现应当使用生成器实现延迟加载方法，以避免一次性将所有文档加载进内存。
5
6 'load' 方法仅供用户方便使用，不应被重写。
7 """
8
9 # 子类不应直接实现此方法。而应实现延迟加载方法。
10 def load(self) -> List[Document]:
11 """将数据加载为 Document 对象。"""
12 return list(self.lazy_load())
13
14 def load_and_split(
15 self, text_splitter: Optional[TextSplitter] = None
16) -> List[Document]:
17 """加载文档并将其分割成块。块以 Document 形式返回。
18 不要重写此方法。它应被视为已弃用！
19 参数:
20 text_splitter: 用于分割文档的 TextSplitter 实例。默认为 RecursiveCharacterTextSplitter。
21 返回:
22 文档列表。
23 """
24
25
26 _text_splitter: TextSplitter = RecursiveCharacterTextSplitter()
27 else:
28 _text_splitter = text_splitter
29 docs = self.load()
30 return _text_splitter.split_documents(docs)
```

**BaseLoader** 把数据加载成 **Documents object**，存到 **Documents** 类中的 **page\_content** 中。

### Document类分析

**Document** 允许用户与文档的内容进行交互，可以查看文档内容。

```
1 class Document(Serializable):
```

```

2 """用于存储文本及其关联元数据的类。"""
3
4 page_content: str
5 """字符串文本。"""
6 metadata: dict = Field(default_factory=dict)
7 """关于页面内容的任意元数据（例如，来源、与其他文档的关系等）。"""
8 type: Literal["Document"] = "Document"
9
10 def __init__(self, page_content: str, **kwargs: Any) -> None:
11 """将 page_content 作为位置参数或命名参数传入。"""
12 super().__init__(page_content=page_content, **kwargs)
13
14 @classmethod
15 def is_lc_serializable(cls) -> bool:
16 """返回此类是否可序列化。"""
17 return True
18
19 @classmethod
20 def get_lc_namespace(cls) -> List[str]:
21 """获取 langchain 对象的命名空间。"""
22 return ["langchain", "schema", "document"]

```

通过 存 + 读的两个基类的抽象，满足不同类型加载器在数据形式上的统一。除此之外，其中的 `metadata` 会根据loader实现的不同写入不同的数据，同样是一个必要的基础属性。

## 3、文档拆分器 Text Splitters

### 3.1 为什么要拆分/分块/切分

当拿到统一的一个Document对象后，接下来需要切分成Chunks。如果不切分，而是考虑作为一个整体的Document对象，会存在两点问题：

1. 假设提问的Query的答案出现在某一个Document对象中，那么将检索到的整个Document对象直接放入Prompt中并 **不是最优的选择**，因为其中一定会 **包含非常多无关的信息**，而无效信息越多，对大模型后续的推理影响越大。
2. 任何一个大模型都存在最大输入的 **Token限制**，如果一个Document非常大，比如一个几百兆的PDF，那么大模型肯定无法容纳如此多的信息。

基于此，一个有效的解决方案就是将完整的Document对象进行 **分块处理 (Chunking)**。无论是在存储还是检索过程中，都将以这些 **块(chunks)** 为基本单位，这样有效地避免内容不相关性问题和超出最大输入限制的问题。

### 3.2 Chunking拆分的策略

**方法1：根据句子切分：**这种方法按照自然句子边界进行切分，以保持语义完整性。

**方法2：按照固定字符数来切分：**这种策略根据特定的字符数量来划分文本，但可能会在不适当的位置切断句子。

**方法3：按固定字符数来切分，结合重叠窗口 (overlapping windows)：**此方法与按字符数切分相似，但通过重叠窗口技术避免切分关键内容，确保信息连贯性。



**方法4：递归字符切分方法：**通过递归字符方式动态确定切分点，这种方法可以根据文档的复杂性和内容密度来调整块的大小。

**方法5：根据语义内容切分：**这种 **高级策略** 依据文本的语义内容来划分块，旨在保持相关信息的集中和完整，适用于需要高度语义保持的应用场景。

第2种方法（按照字符数切分）和第3种方法（按固定字符数切分结合重叠窗口）主要基于字符进行文本的切分，而不考虑文章的实际内容和语义。这种方式虽简单，但可能会导致 **主题或语义上的断裂**。

相对而言，第4种递归方法更加灵活和高效，它结合了固定长度切分和语义分析。通常是 **首选策略**，因为它能够更好地确保每个段落包含一个完整的主题。

而第5种方法，基于语义的分割虽然能精确地切分出完整的主题段落，但这种方法效率较低。它需要运行复杂的分段算法（segmentation algorithm），**处理速度较慢**，并且 **段落长度可能极不均匀**（有的主题段落可能很长，而有的则较短）。因此，尽管它在某些需要高精度语义保持的场景下有其应用价值，但并不 **适合所有情况**。

这些方法各有优势和局限，选择适当的分块策略取决于具体的应用需求和预期的检索效果。接下来我们依次尝试用常规手段应该如何实现上述几种方法的文本切分。

### 3.3 TextSplitter 源码分析

```
1 class TextSplitter(BaseDocumentTransformer, ABC):
2 """用于将文本分割成块的接口。"""
3
4 def __init__(
5 self,
6 chunk_size: int = 4000,
7 chunk_overlap: int = 200, #
8 length_function: Callable[[str], int] = len,
9 keep_separator: bool = False,
10 add_start_index: bool = False,
11 strip_whitespace: bool = True,
12) -> None:
13 """
14 创建一个新的文本分割器。
15
16 参数:
17 chunk_size: 返回块的最大尺寸，单位是字符数。默认值为4000（由长度函数测量）
18
19 chunk_overlap: 相邻两个块之间的字符重叠数，避免信息在边界处被切断而丢失。默认值为200，
20 通常会设置为chunk_size的10% - 20%。
21
22 length_function: 用于测量给定块字符数的函数。默认赋值为len函数。len函数在Python中按
23 Unicode字符计数，所以一个汉字、一个英文字母、一个符号都算一个字符。
24
25 keep_separator: 是否在块中保留分隔符，默认值为False
26
27 add_start_index: 如果为`True`，则在元数据中包含块的起始索引。默认值为False
28
29 strip_whitespace: 如果为`True`，则从每个文档的开始和结束处去除空白字符。默认值为True
30 """
31 if chunk_overlap > chunk_size:
```

```

30 raise ValueError(
31 f"重叠大小 ({chunk_overlap}) 大于块大小 ({chunk_size}) 。 "
32)
33 self.chunk_size = chunk_size
34 self.chunk_overlap = chunk_overlap
35 self.length_function = length_function
36 self.keep_separator = keep_separator
37 self.add_start_index = add_start_index
38 self.strip_whitespace = strip_whitespace
39
40 @abstractmethod
41 def split_text(self, text: str) -> List[str]:
42 """将文本分割成多个字符串。具体实现由子类决定"""
43
44
45 def create_documents(
46 self, texts: List[str], metadatas: Optional[List[dict]] = None
47) -> List[Document]:
48 """基于文本列表创建Documents对象。作用是将普通的字符串列表对象转化成Document列表
49 对象，同时考虑切分"""
50 _metadatas = metadatas or [{}] * len(texts)
51 documents = []
52 for i, text in enumerate(texts):
53 index = 0
54 previous_chunk_len = 0
55 for chunk in self.split_text(text):
56 metadata = copy.deepcopy(_metadatas[i])
57 if self.add_start_index:
58 offset = index + previous_chunk_len - self.chunk_overlap
59 index = text.find(chunk, max(0, offset))
60 metadata["start_index"] = index
61 previous_chunk_len = len(chunk)
62 new_doc = Document(page_content=chunk, metadata=metadata)
63 documents.append(new_doc)
64
65 def return documents
66
67
68 def split_documents(self, documents: Iterable[Document]) -> List[Document]:
69 """分割文档。"""
70 texts, metadatas = [], []
71 for doc in documents:
72 texts.append(doc.page_content)
73 metadatas.append(doc.metadata)
74 return self.create_documents(texts, metadatas=metadatas)
75
76 @classmethod
77 def from_huggingface_tokenizer(cls, tokenizer: Any, **kwargs: Any) -> TextSplitter:
78 """使用 HuggingFace的分词器来计数长度的文本分割器。"""
79 try:
80 from transformers import PreTrainedTokenizerBase
81
82 if not isinstance(tokenizer, PreTrainedTokenizerBase):
83 raise ValueError(

```

```

84 "Tokenizer received was not an instance of PreTrainedTokenizerBase"
85)
86
87 def _huggingface_tokenizer_length(text: str) -> int:
88 return len(tokenizer.tokenize(text))
89
90 except ImportError:
91 raise ValueError(
92 "Could not import transformers python package. "
93 "Please install it with `pip install transformers`."
94)
95 return cls(length_function=_huggingface_tokenizer_length, **kwargs)
96
97 @classmethod
98 def from_tiktoken_encoder(
99 cls: Type[TS],
100 encoding_name: str = "gpt2",
101 model_name: Optional[str] = None,
102 allowed_special: Union[Literal["all"], AbstractSet[str]] = set(),
103 disallowed_special: Union[Literal["all"], Collection[str]] = "all",
104 **kwargs: Any,
105) -> TS:
106 """使用 TikToken 编码器来计数长度的文本分割器。"""
107 #实现细节: 略
108
109
110 def transform_documents(
111 self, documents: Sequence[Document], **kwargs: Any
112) -> Sequence[Document]:
113 """通过分割它们来转换文档序列。"""
114 return self.split_documents(list(documents))
115

```

小结：几个常用的文档切分器的方法的调用

```

1 #方式1: 传入的参数类型: 字符串; 返回值类型: List[str]
2 split_text(xxx)
3
4 #方式2: 传入的参数类型: List[str]; 返回值类型: List[Document]
5 create_documents(xxx) #底层调用了split_text(xxx)
6
7 #方式3: 传入的参数类型: List[Document]; 返回值类型: List[Document]
8 split_documents(xxx) #底层调用了create_documents()

```

此外，这里提供了一个可视化展示文本如何分割的工具，<https://chunkviz.up.railway.app/>

## 3.4 具体实现

LangChain提供了许多不同类型的文档切分器

官网地址: [https://python.langchain.com/api\\_reference/text\\_splitters/index.html](https://python.langchain.com/api_reference/text_splitters/index.html)

### 3.3.1 CharacterTextSplitter: Split by character

参数情况说明:

- **chunk\_size** : 每个切块的最大token数量, 默认值为4000。
- **chunk\_overlap** : 相邻两个切块之间的最大重叠token数量, 默认值为200。
- **separator** : 分割使用的分隔符, 默认值为"\n\n"。
- **length\_function** : 用于计算切块长度的方法。默认赋值为父类TextSplitter的len函数。

举例1: 字符串文本的分割

```
1 # 1.导入相关依赖
2 from langchain.text_splitter import CharacterTextSplitter
3
4 # 2.示例文本
5 text = """
6 LangChain 是一个用于开发由语言模型驱动的应用程序的框架的。它提供了一套工具和抽象, 使开发者
7 能够更容易地构建复杂的应用程序。
8 """
9 # 3.定义字符分割器
10 splitter = CharacterTextSplitter(
11 chunk_size=50, # 每块大小
12 chunk_overlap=5, # 块与块之间的重复字符数
13 #length_function=len,
14 separator= "" # 设置为空字符串时, 表示禁用分隔符优先
15)
16
17 # 4.分割文本
18 texts = splitter.split_text(text)
19
20 # 5.打印结果
21 for i, chunk in enumerate(texts):
22 print(f"块 {i+1}:长度: {len(chunk)}")
23 print(chunk)
24 print("-" * 50)
```

```
1 块 1:长度: 49
2 LangChain 是一个用于开发由语言模型驱动的应用程序的框架的。它提供了一套工具和抽象, 使开
3 发
4 块 2:长度: 22
5 象, 使开发者能够更容易地构建复杂的应用程序。
6 -----
```

说明: 若必须禁用分隔符 (如处理无空格文本), 需容忍实际块长略小于 **chunk\_size** (尤其对中文)

## 举例2：指定分割符

```
1 # 1.导入相关依赖
2 from langchain.text_splitter import CharacterTextSplitter
3
4 # 2.定义要分割的文本
5 text = "这是一个示例文本啊。我们将使用CharacterTextSplitter将其分割成小块。分割基于字符数。"
6
7 # text = ""
8 # LangChain 是一个用于开发由语言模型。驱动的应用程序的框架的。它提供了一套工具和抽象。使开
9 # 发者能够更容易地构建复杂的应用程序。
10 # ""
11
12 # 3.定义分割器实例
13 text_splitter = CharacterTextSplitter(
14 chunk_size=30, # 每个块的最大字符数
15 chunk_overlap=5, # 块之间的重叠字符数
16 separator="。"; # 按句号分割
17)
18
19 # 4.开始分割
20 chunks = text_splitter.split_text(text)
21
22 # 5.打印效果
23 for i, chunk in enumerate(chunks):
24 print(f"块 {i + 1}: 长度: {len(chunk)}")
25 print(chunk)
26 print("-"*50)
```

```
1 Created a chunk of size 33, which is longer than the specified 30
2
3 块 1: 长度: 9
4 这是一个示例文本啊
5 -----
6 块 2: 长度: 33
7 我们将使用CharacterTextSplitter将其分割成小块
8 -----
9 块 3: 长度: 7
10 分割基于字符数
11 -----
```

**注意：无重叠。**

**separator优先原则：**当设置了 `separator`（如"。"），分割器会首先尝试在分隔符处分割，然后再考虑 `chunk_size`。这是为了避免在句子中间硬性切断。这种设计是为了：

1. 优先保持语义完整性（不切断句子）
2. 避免产生无意义的碎片（如半个单词/不完整句子）
3. 如果 `chunk_size` 比片段小，无法拆分片段，导致 `overlap` 失效。

4. `chunk_overlap`仅在合并后的片段之间生效（如果 `chunk_size` 足够大）。如果没有合并的片段，则 `overlap`失效。见举例3。

举例3：指定分割符

**注意：有重叠。**此时，文本“这是第二段内容。”的token正好就是8。

```
1 # 1.导入相关依赖
2 from langchain.text_splitter import CharacterTextSplitter
3
4 # 2.定义要分割的文本
5 text = "这是第一段文本。这是第二段内容。最后一段结束。"
6
7 # 3.定义字符分割器
8 text_splitter = CharacterTextSplitter(
9 separator="。",
10 chunk_size=20,
11 chunk_overlap=8,
12 keep_separator=True #chunk中是否保留切割符
13)
14
15 # 4.分割文本
16 chunks = text_splitter.split_text(text)
17
18 # 5.打印结果
19 for i,chunk in enumerate(chunks):
20 print(f"块 {i + 1}:长度: {len(chunk)}")
21 print(chunk)
22 print("-"*50)
```

```
1 块 1:长度: 15
2 这是第一段文本。这是第二段内容
3 -----
4 块 2:长度: 16
5 。这是第二段内容。最后一段结束。
6 -----
```

### 3.3.2 RecursiveCharacterTextSplitter: 最常用

文档切分器中较常用的是 `RecursiveCharacterTextSplitter` (递归字符文本切分器)，遇到 特定字符 时进行分割。默认情况下，它尝试进行切割的字符包括 `["\n\n", "\n", " ", ""]`。

具体为：根据第一个字符进行切块，但如果任何切块太大，则会继续移动到下一个字符继续切块，以此类推。

此外，还可以考虑添加，。等分割字符。

**特点：**

- **保留上下文：**优先在自然语言边界（如段落、句子结尾）处分割，减少信息碎片化。

- **智能分段**：通过递归尝试多种分隔符，将文本分割为大小接近chunk\_size的片段。
- **灵活适配**：适用于多种文本类型（代码、Markdown、普通文本等），是LangChain中 **最通用** 的文本拆分器。

此外，还可以指定的参数包括：

- **chunk\_size**：同TextSplitter（父类）。
- **chunk\_overlap**：同TextSplitter（父类）。
- **length\_function**：同TextSplitter（父类）。
- **add\_start\_index**：同TextSplitter（父类）。

举例1：使用split\_text()方法演示

```

1 # 1.导入相关依赖
2 from langchain.text_splitter import RecursiveCharacterTextSplitter
3
4 # 2.定义RecursiveCharacterTextSplitter分割器对象
5 text_splitter = RecursiveCharacterTextSplitter(
6 chunk_size=10,
7 chunk_overlap=0,
8 add_start_index=True,
9)
10
11 # 3.定义拆分的内容
12 text= "LangChain框架特性\n\n多模型集成(GPT/Claude)\n\n记忆管理功能\n\n链式调用设计。文档分析场
 景示例：需要处理PDF/Word等格式。"
13
14 # 4.拆分器分割
15 paragraphs = text_splitter.split_text(text)
16
17 for para in paragraphs:
18 print(para)
19 print('-----')

```

```

1 LangChain框
2 -----
3 架特性
4 -----
5 多模型集成(GPT
6 -----
7 /Claude)
8 -----
9 记忆管理功能
10 -----
11 链式调用设计。文档
12 -----
13 分析场景示例：需要处
14 -----
15 理PDF/Word等
16 -----
17 格式。
18 -----

```



举例2：使用create\_documents()方法演示，传入字符串列表，返回Document对象列表

```
1 # 1.导入相关依赖
2 from langchain.text_splitter import RecursiveCharacterTextSplitter
3
4 # 2.定义RecursiveCharacterTextSplitter分割器对象
5 text_splitter = RecursiveCharacterTextSplitter(
6 chunk_size=10,
7 chunk_overlap=0,
8 add_start_index=True,
9)
10
11 # 3.定义分割的内容
12 # text="LangChain框架特性\n\n多模型集成(GPT/Claude)\n\n记忆管理功能\n\n链式调用设计。文档分析场
 场景示例：需要处理PDF/Word等格式。"
13
14 list=["LangChain框架特性\n\n多模型集成(GPT/Claude)\n\n记忆管理功能\n\n链式调用设计。文档分析场
 场景示例：需要处理PDF/Word等格式。"]
15
16 # 4.分割器分割
17 # create_documents(): 形参是字符串列表，返回值是Document的列表
18 paragraphs = text_splitter.create_documents(list)
19
20
21 for para in paragraphs:
22 print(para)
23 print('-----')
```

```
1 page_content='LangChain框' metadata={'start_index': 0}
2 -----
3 page_content='架特性' metadata={'start_index': 10}
4 -----
5 page_content='多模型集成(GPT' metadata={'start_index': 15}
6 -----
7 page_content='/Claude)' metadata={'start_index': 24}
8 -----
9 page_content='记忆管理功能' metadata={'start_index': 33}
10 -----
11 page_content='链式调用设计。文档' metadata={'start_index': 40}
12 -----
13 page_content='分析场景示例：需要处' metadata={'start_index': 49}
14 -----
15 page_content='理PDF/Word等' metadata={'start_index': 59}
16 -----
17 page_content='格式。' metadata={'start_index': 69}
18 -----
```

## 逐步分割过程

### 第一阶段：顶级分割（按 \n\n）

## 1. 首次分割:

```
1 text.split("\n\n") →
2 [
3 "LangChain框架特性",
4 "多模型集成(GPT/Claude)\n记忆管理功能\n链式调用设计。文档分析场景示例: 需要处理
 PDF/Word等格式。"
5]
```

- 第一部分长度: 13字符 > 10 → 需要继续分割
- 第二部分长度: 79字符 > 10 → 需要继续分割

## 第二阶段: 递归分割第一部分 "LangChain框架特性"

1. 尝试 `\n`: 无匹配
2. 尝试  (空格):
  - 检查字符串: "LangChain框架特性" (无空格)
3. 回退到 `""` (字符级分割):

```
1 list("LangChain框架特性") →
2 ['L','a','n','g','C','h','a','i','n','框','架','特','性']
```

- 前10字符: "LangChain框"
- 剩余部分: "架特性"

## 第三阶段: 递归分割第二部分 (长段落)

### 1. 按 `\n` 分割:

```
1 "多模型集成(GPT/Claude)\n记忆管理功能\n链式调用设计。文档..." .split("\n") →
2 [
3 "多模型集成(GPT/Claude)", # 17字符
4 "记忆管理功能", # 6字符
5 "链式调用设计。文档分析场景示例: 需要处理PDF/Word等格式。" # 36字符
6]
```

- 第1块: 17字符 > 10 → 继续分割
  - 第2块: 6字符 ≤ 10 → 直接保留
  - 第3块: 36字符 > 10 → 继续分割
2. 分割 "多模型集成(GPT/Claude)":
    - 尝试 : 无空格
    - 回退到 `""`:
      - 前10字符: "多模型集成(GPT"
      - 剩余7字符: " /Claude)"
  3. 分割 "链式调用设计。文档分析场景示例: 需要处理PDF/Word等格式。":
    - 尝试 : 无空格
    - 回退到 `""`:

■ 按10字符分段：

```
1 "链式调用设计。文档分析场景示例：需要处理PDF/Word等格式。"
2 →
3 [
4 "链式调用设计。文档",
5 "分析场景示例：需要处",
6 "理PDF/Word等",
7 "格式。"
8]
```

举例3：使用create\_documents()方法演示，将本地文件内容加载成字符串，进行拆分

```
1 # 1.导入相关依赖
2 from langchain_text_splitters import RecursiveCharacterTextSplitter
3
4 # 2.打开.txt文件
5 with open("asset/load/08-ai.txt", encoding="utf-8") as f:
6 state_of_the_union = f.read() #返回的是字符串
7
8 # 3.定义RecursiveCharacterTextSplitter (递归字符分割器)
9 text_splitter = RecursiveCharacterTextSplitter(
10 chunk_size=100,
11 chunk_overlap=20,
12 #chunk_overlap=0,
13 length_function=len
14)
15
16 # 4.分割文本
17 texts = text_splitter.create_documents([state_of_the_union])
18
19 # 5.打印分割文本
20 for text in texts:
21 print(f"👁️ {text.page_content}")
22
```

- 1 👁️ 人工智能（AI）是什么？
- 2 👁️ 人工智能（Artificial
- 3 👁️ Intelligence，简称AI）是指由计算机系统模拟人类智能的技术，使其能够执行通常需要人类认知能力的任务，如学习、推理、决策和语言理解。AI的核心目标是让机器具备感知环境、处理信息并自主行动的
- 4 👁️ 让机器具备感知环境、处理信息并自主行动的能力。
- 5 👁️ 1. AI的技术基础
- 6 AI依赖多种关键技术：
- 7
- 8 机器学习（ML）：通过算法让计算机从数据中学习规律，无需显式编程。例如，推荐系统通过用户历史行为预测偏好。
- 9 👁️ 深度学习：基于神经网络的机器学习分支，擅长处理图像、语音等复杂数据。AlphaGo击败围棋冠军便是典型案例。

10  
11 自然语言处理（NLP）：使计算机理解、生成人类语言，如ChatGPT的对话能力。  
12 📍 2. AI的应用场景  
13 AI已渗透到日常生活和各行各业：  
14  
15 医疗：辅助诊断（如AI分析医学影像）、药物研发加速。  
16  
17 交通：自动驾驶汽车通过传感器和AI算法实现安全导航。  
18 📍 金融：欺诈检测、智能投顾（如风险评估模型）。  
19  
20 教育：个性化学习平台根据学生表现调整教学内容。  
21  
22 3. AI的挑战与未来  
23 尽管前景广阔，AI仍面临问题：  
24 📍 伦理争议：数据隐私、算法偏见（如招聘AI歧视特定群体）。  
25  
26 就业影响：自动化可能取代部分人工岗位，但也会创造新职业。  
27  
28 技术瓶颈：通用人工智能（AGI）尚未实现，当前AI仅擅长特定任务。  
29 📍 未来，AI将与人类协作而非替代：医生借助AI提高诊断效率，教师利用AI定制课程。其发展需平衡技术创新与社会责任，确保技术造福全人类。

举例4：使用split\_documents()方法演示，利用PDFLoader加载文档，对文档的内容用递归切割器切割

```
1 # 1.导入相关依赖
2 from langchain_community.document_loaders import PyPDFLoader
3 from langchain.text_splitter import RecursiveCharacterTextSplitter
4
5 # 2.定义PyPDFLoader加载器
6 loader = PyPDFLoader("./asset/load/02-load.pdf")
7
8 # 3.加载和切割文档对象
9 docs = loader.load() # 返回Document对象构成的list
10 # print(f"第0页: \n{docs[0]}")
11
12 # 4.定义切割器
13 text_splitter = RecursiveCharacterTextSplitter(
14 chunk_size=200,
15 #chunk_size=120,
16 chunk_overlap=0,
17 # chunk_overlap=100,
18 length_function=len,
19 add_start_index=True,
20)
21
22 # 5.对pdf内容进行切割得到文档对象
23 paragraphs = text_splitter.split_documents(docs)
24 #paragraphs = text_splitter.create_documents([text])
25 for para in paragraphs:
26 print(para.page_content)
27 print('-----')
```

```
1 "他的车，他的命！他忽然想起来，一年，二年，至少有三四年；一滴汗，两滴汗，不
2 知道多少万滴汗，才挣出那辆车。从风里雨里的咬牙，从饭里茶里的自苦，才赚出那辆车。
3 那辆车是他的一切挣扎与困苦的总结果与报酬，像身经百战的武士的一颗徽章。……他老想
4 着远远的一辆车，可以使他自由，独立，像自己的手脚的那么一辆车。"
5
6 "他吃，他喝，他嫖，他赌，他懒，他狡猾，因为他没了心，他的心被人家摘了去。他
7 -----
8 只剩下那个高大的肉架子，等着溃烂，预备着到乱死岗子去。……体面的、要强的、好梦想
9 的、利己的、个人的、健壮的、伟大的祥子，不知陪着人家送了多少回殡；不知道何时何地
10 会埋起他自己来，埋起这堕落的、自私的、不幸的、社会病胎里的产儿，个人主义的末路鬼！
11 "
12 -----
```

### 举例5：自定义分隔符

有些书写系统没有单词边界，例如中文、日文和泰文。使用默认分隔符列表["\n\n", "\n", " ", ""]分割文本可能导致单词错误的分割。为了保持单词在一起，你可以自定义分割字符，覆盖分隔符列表以包含额外的标点符号。

```
1 text_splitter = RecursiveCharacterTextSplitter(
2 chunk_size=200,
3 chunk_overlap=20, # 增加重叠字符
4 separators=["\n\n", "\n", "。", "！", "？", "……", "；", "，", "。"], # 添加中文标点
5 length_function=len,
6 keep_separator=True # 保留句尾标点（如……），避免切割后丢失语气和逻辑
7)
```

效果：算法优先在句号、省略号处切割，保持句子完整性。

### 3.3.3 TokenTextSplitter/CharacterTextSplitter: Split by tokens

当我们将文本拆分为块时，除了字符以外，还可以：**按Token的数量分割**（而非字符或单词数），将长文本切分成多个小块。

#### 什么是Token？

- 对模型而言，Token是文本的最小处理单位。例如：
  - 英文："hello" → 1个Token，"ChatGPT" → 2个Token（"Chat" + "GPT"）。
  - 中文："人工智能" → 可能拆分为2-3个Token（取决于分词器）。

#### 为什么按Token分割？

- 语言模型对输入长度的限制是基于Token数（如GPT-4的8k/32k Token上限），直接按字符或单词分割可能导致实际Token数超限。（确保每个文本块不超过模型的Token上限）
- 大语言模型(LLM)通常是以token的数量作为其计量(或收费)的依据，所以采用token分割也有助于我们在使用时更方便的控制成本。

#### TokenTextSplitter 使用说明：

- 核心依据：Token数量 + 自然边界。（TokenTextSplitter 严格按照 token 数量进行分割，但同时会优先在自然边界（如句尾）处切断，以尽量保证语义的完整性。）

- 优点：与LLM的Token计数逻辑一致，能尽量保持语义完整
- 缺点：对非英语或特定领域文本，Token化效果可能不佳
- 典型场景：需要精确控制Token数输入LLM的场景

#### 举例1：使用TokenTextSplitter

```
1 # 1.导入相关依赖
2 from langchain_text_splitters import TokenTextSplitter
3
4 # 2.初始化 TokenTextSplitter
5 text_splitter = TokenTextSplitter(
6 chunk_size=33, #最大 token 数为 32
7 chunk_overlap=0, #重叠 token 数为 0
8 encoding_name="cl100k_base", #使用 OpenAI 的编码器,将文本转换为 token 序列
9
10)
11 # 3.定义文本
12 text = "人工智能是一个强大的开发框架。它支持多种语言模型和工具链。人工智能是指通过计算机程序模拟人类智能的一门科学。自20世纪50年代诞生以来，人工智能经历了多次起伏。"
13
14 # 4.开始切割
15 texts = text_splitter.split_text(text)
16
17 # 打印分割结果
18 print(f"原始文本被分割成了 {len(texts)} 个块:")
19 for i, chunk in enumerate(texts):
20 print(f"块 {i+1}: 长度: {len(chunk)} 内容: {chunk}")
21 print("-" * 50)
```

```
1 原始文本被分割成了 3 个块:
2 块 1: 长度: 29 内容: 人工智能是一个强大的开发框架。它支持多种语言模型和工具链。
3 -----
4 块 2: 长度: 32 内容: 人工智能是指通过计算机程序模拟人类智能的一门科学。自20世纪50
5 -----
6 块 3: 长度: 19 内容: 年代诞生以来，人工智能经历了多次起伏。
7 -----
```

#### 为什么会出现这样的分割？

**1、第一块 (29字符)：**内容是一个完整的句子，以句号结尾。TokenTextSplitter识别到这是一个自然的语义边界，即使这里的 token 数量可能尚未达到 33，它也选择在此处切割，以保证第一块语义的完整性。

**2、第二块 (32字符)：**内容包含了另一个完整句子 “人工智能是指...一门科学。” 以及下一句的开头 “自20世纪50”。分割器在处理完第一个句子的 token 后，可能 token 数量已经接近 `chunk_size`，于是在下一个自然边界（这里是句号）之后继续读取了少量 token（“自20世纪50”），直到非常接近 33 token 的限制。

**注意：**“50” 之后被切断，是因为编码器很可能将 “50” 识别为一个独立的 token，而 “年代” 是另一个 token。为了保证 token 的完整性，它不会在 “50” 字符中间切断。

**3、第三块 (19字符)**：是第二块中断内容的剩余部分，形成了一个较短的块。这是因为剩余内容本身的 token 数量就较少。

特别注意：**字符长度不等于 Token 数量。**

举例2：使用CharacterTextSplitter

```
1 # 1.导入相关依赖
2 from langchain_text_splitters import CharacterTextSplitter
3 import tiktoken # 用于计算Token数量
4
5
6 # 2.定义通过Token切割器
7 text_splitter = CharacterTextSplitter.from_tiktoken_encoder(
8 encoding_name="cl100k_base", # 使用 OpenAI 的编码器
9 chunk_size=18,
10 chunk_overlap=0,
11 separator="。", # 指定中文句号为分隔符
12 keep_separator=False, # chunk中是否保留分隔符
13)
14 # 3.定义文本
15 text = "人工智能是一个强大的开发框架。它支持多种语言模型和工具链。今天天气很好，想出去踏青。但是又比较懒不想出去，怎么办"
16
17 # 4.开始切割
18 texts = text_splitter.split_text(text)
19 print(f"分割后的块数: {len(texts)}")
20
21 # 5.初始化tiktoken编码器 (用于Token计数)
22 encoder = tiktoken.get_encoding("cl100k_base") # 确保与CharacterTextSplitter的
23 encoding_name一致
24
25 # 6.打印每个块的Token数和内容
26 for i, chunk in enumerate(texts):
27 tokens = encoder.encode(chunk) # 现在encoder已定义
28 print(f"块 {i + 1}: {len(tokens)} Token\n内容: {chunk}\n")
```

```
1 分割后的块数: 4
2 块 1: 17 Token
3 内容: 人工智能是一个强大的开发框架
4
5 块 2: 14 Token
6 内容: 它支持多种语言模型和工具链
7
8 块 3: 18 Token
9 内容: 今天天气很好，想出去踏青
10
11 块 4: 21 Token
12 内容: 但是又比较懒不想出去，怎么办
```



3.3.4 SemanticChunker: 语义分块

SemanticChunking（语义分块）是 LangChain 中一种更高级的文本分割方法，它超越了传统的基于字符或固定大小的分块方式，而是根据 文本的语义结构 进行智能分块，使每个分块保持 语义完整性，从而提高检索增强生成(RAG)等应用的效果。

语义分割 vs 传统分割

特性	语义分割 (SemanticChunker)	传统字符分割 (RecursiveCharacter)
分割依据	嵌入向量相似度	固定字符/换行符
语义完整性	☑ 保持主题连贯	✗ 可能切断句子逻辑
计算成本	✗ 高 (需嵌入模型)	☑ 低
适用场景	需要高语义一致性的任务	简单文本预处理

举例：

```
1 from langchain_experimental.text_splitter import SemanticChunker
2 from langchain_openai.embeddings import OpenAIEmbeddings
3 import os
4 import dotenv
5
6 dotenv.load_dotenv()
7
8 # 加载文本
9 with open("asset/load/09-ai1.txt", encoding="utf-8") as f:
10 state_of_the_union = f.read() #返回字符串
11
12 # 获取嵌入模型
13 os.environ['OPENAI_API_KEY'] = os.getenv("OPENAI_API_KEY1")
14 os.environ['OPENAI_BASE_URL'] = os.getenv("OPENAI_BASE_URL")
15 embed_model = OpenAIEmbeddings(
16 model="text-embedding-3-large"
17)
18
19 # 获取切割器
20 text_splitter = SemanticChunker(
21 embeddings=embed_model,
22 breakpoint_threshold_type="percentile",#断点阈值类型: 字面值["百分位数", "标准差", "四分位距", "梯度"] 选其一
23 breakpoint_threshold_amount=65.0 #断点阈值数量 (极低阈值 → 高分割敏感度)
24)
25
26 # 切分文档
27 docs = text_splitter.create_documents(texts = [state_of_the_union])
28 print(len(docs))
29 for doc in docs:
```

1 4

2 📄 文档 page\_content='人工智能综述：发展、应用与未来展望'

3

4 摘要

5 人工智能（Artificial Intelligence, AI）作为计算机科学的一个重要分支，近年来取得了突飞猛进的发展。本文综述了人工智能的发展历程、核心技术、应用领域以及未来发展趋势。通过对人工智能的定义、历史背景、主要技术（如机器学习、深度学习、自然语言处理等）的详细介绍，探讨了人工智能在医疗、金融、教育、交通等领域的应用，并分析了人工智能发展过程中面临的挑战与机遇。最后，本文对人工智能的未来发展进行了展望，提出了可能的突破方向。

6

7 1. 引言

8 人工智能是指通过计算机程序模拟人类智能的一门科学。自20世纪50年代诞生以来，人工智能经历了多次起伏，近年来随着计算能力的提升和大数据的普及，人工智能技术取得了显著的进展。人工智能的应用已经渗透到日常生活的方方面面，从智能手机的语音助手到自动驾驶汽车，从医疗诊断到金融分析，人工智能正在改变着人类社会的运行方式。

9

10 2.:

11 📄 文档 page\_content='人工智能的发展历程'

12 2.1 早期发展

13 人工智能的概念最早可以追溯到20世纪50年代。1956年，达特茅斯会议（Dartmouth Conference）被认为是人工智能研究的正式开端。在随后的几十年里，人工智能研究经历了多次高潮与低谷。早期的研究主要集中在符号逻辑和专家系统上，但由于计算能力的限制和算法的不足，进展缓慢。

14 2.2 机器学习的兴起

15 20世纪90年代，随着统计学习方法的引入，机器学习逐渐成为人工智能研究的主流。支持向量机（SVM）、决策树、随机森林等算法在分类和回归任务中取得了良好的效果。这一时期，机器学习开始应用于数据挖掘、模式识别等领域。

16 2.3 深度学习的突破

17 2012年，深度学习在图像识别领域取得了突破性进展，标志着人工智能进入了一个新的阶段。深度学习通过多层神经网络模拟人脑的工作方式，能够自动提取特征并进行复杂的模式识别。卷积神经网络（CNN）、循环神经网络（RNN）和长短期记忆网络（LSTM）等深度学习模型在图像处理、自然语言处理、语音识别等领域取得了显著成果。

18

19 3. 人工智能的核心技术

20 3.1 机器学习

21 机器学习是人工智能的核心技术之一，通过算法使计算机从数据中学习并做出决策。常见的机器学习算法包括监督学习、无监督学习和强化学习。监督学习通过标记数据进行训练，无监督学习则从未标记数据中寻找模式，强化学习则通过与环境交互来优化决策。

22 3.2 深度学习

23 深度学习是机器学习的一个子领域，通过多层神经网络进行特征提取和模式识别。深度学习在图像识别、自然语言处理、语音识别等领域取得了显著成果。常见的深度学习模型包括卷积神经网络（CNN）、循环神经网络（RNN）和长短期记忆网络（LSTM）。

24 3.3 自然语言处理

25 自然语言处理（NLP）是人工智能的一个重要分支，致力于使计算机能够理解和生成人类语言。NLP技术广泛应用于机器翻译、情感分析、文本分类等领域。近年来，基于深度学习的NLP模型（如BERT、GPT）在语言理解任务中取得了突破性进展。

26 3.4 计算机视觉

27 计算机视觉是人工智能的另一个重要分支，致力于使计算机能够理解和处理图像和视频。计算机视觉技术广泛应用于图像识别、目标检测、人脸识别等领域。深度学习模型（如CNN）在计算机视觉任务中取得了显著成果。

28

29 4. 人工智能的应用领域

30 4.1 医疗健康

31 人工智能在医疗健康领域的应用包括疾病诊断、药物研发、个性化医疗等。通过分析医学影像和患者数据，人工智能可以帮助医生更准确地诊断疾病，提高治疗效果。

32 4.2 金融

33 人工智能在金融领域的应用包括风险评估、欺诈检测、算法交易等。通过分析市场数据和交易记录，人工智能可以帮助金融机构做出更明智的决策，提高运营效率。

34 4.3 教育

35 人工智能在教育领域的应用包括个性化学习、智能辅导、自动评分等。通过分析学生的学习数据，人工智能可以为学生提供个性化的学习建议，提高学习效果。

36 4.4 交通

37 人工智能在交通领域的应用包括自动驾驶、交通管理、智能导航等。通过分析交通数据和路况信息，人工智能可以帮助优化交通流量，提高交通安全。

38

39 5. 人工智能的挑战与机遇

40 5.1 挑战

41 人工智能发展过程中面临的主要挑战包括数据隐私、算法偏见、安全性问题等。数据隐私问题涉及到个人数据的收集和使用，算法偏见问题则涉及到算法的公平性和透明度，安全性问题则涉及到人工智能系统的可靠性和稳定性。

42 5.2 机遇

43 尽管面临挑战，人工智能的发展也带来了巨大的机遇。人工智能技术的进步将推动各行各业的创新，提高生产效率，改善生活质量。未来，人工智能有望在更多领域取得突破，为人类社会带来更多的便利和福祉。

44

45 6.:

46 🔍 文档 page\_content='未来展望'

47 6.1 技术突破

48 未来，人工智能技术有望在以下几个方面取得突破：一是算法的优化和创新，提高模型的效率和准确性；二是计算能力的提升，支持更复杂的模型和更大规模的数据处理；三是跨学科研究的深入，推动人工智能与其他领域的融合。

49 6.2 应用拓展

50 随着技术的进步，人工智能的应用领域将进一步拓展。未来，人工智能有望在更多领域发挥重要作用，如环境保护、能源管理、智能制造等。人工智能将成为推动社会进步的重要力量。

51

52 7.:

53 🔍 文档 page\_content='结论'

54 人工智能作为一门快速发展的科学，正在改变着人类社会的运行方式。通过不断的技术创新和应用拓展，人工智能将为人类社会带来更多的便利和福祉。然而，人工智能的发展也面临着诸多挑战，需要社会各界共同努力，推动人工智能的健康发展。':

关于参数的说明:

1. breakpoint\_threshold\_type (断点阈值类型)

- 作用：定义文本语义边界的检测算法，决定何时分割文本块。
- 可选值及原理：

类型	原理说明	适用场景
percentile	计算相邻句子嵌入向量的余弦距离，取 <b>距离分布的第N百分位值</b> 作为阈值，高于此值则分割	常规文本（如文章、报告）
standard_deviation	以 <b>均值 + N倍标准差</b> 为阈值，识别语义突变点	语义变化剧烈的文档（如技术手册）
interquartile	用 <b>四分位距（IQR）</b> 定义异常值边界，超过则分割	长文档（如书籍）
gradient	基于 <b>嵌入向量变化的梯度</b> 检测分割点（需自定义实现）	实验性需求

2. `breakpoint_threshold_amount`（断点阈值量）

- **作用：**控制分割的**粒度敏感度**，值越小分割越细（块越多），值越大分割越粗（块越少）。
- 取值范围与示例：
  - `percentile` 模式：0.0~100.0，用户代码设 `65.0` 表示仅当余弦距离 > 所有距离中最低的 65.0%值时分割。默认值是：95.0，兼顾语义完整性与检索效率。值过小（比如0.1），会产生大量小文本块，过度分割可能导致上下文断裂。
  - `standard_deviation` 模式：浮点数（如 `1.5` 表示均值+1.5倍标准差）。
  - `interquartile` 模式：倍数（如 `1.5` 是IQR标准值）。

3.3.5 其它拆分器

类型1: HTMLHeaderTextSplitter: Split by HTML header

HTMLHeaderTextSplitter是一种专门用于处理HTML文档的文本分割方法，它根据HTML的 **标题标签**（如<h1>、<h2>等）将文档划分为逻辑分块，同时保留标题的层级结构信息。

举例：

```
1 # 1.导入相关依赖
2 from langchain.text_splitter import HTMLHeaderTextSplitter
3
4 # 2.定义HTML文件
5 html_string = """
6 <!DOCTYPE html>
7 <html>
8 <body>
9 <div>
10 <h1>欢迎来到尚硅谷! </h1>
11 <p>尚硅谷是专门培训IT技术方向</p>
12 <div>
13 <h2>尚硅谷老师简介</h2>
14 <p>尚硅谷老师拥有多年教学经验，都是从一线互联网下来</p>
15 <h3>尚硅谷北京校区</h3>
16 <p>北京校区位于宏福科技园区</p>
17 </div>
18 </div>
```

```

19 </body>
20 </html>
21 """
22
23 # 4.用于指定要根据哪些HTML标签来分割文本
24 headers_to_split_on = [
25 ("h1", "标题1"),
26 ("h2", "标题2"),
27 ("h3", "标题3"),
28]
29
30 # 5.定义HTMLHeaderTextSplitter分割器
31 html_splitter = HTMLHeaderTextSplitter(headers_to_split_on=headers_to_split_on)
32
33 # 6.分割器分割
34 html_header_splits = html_splitter.split_text(html_string)
35
36 html_header_splits

```

```

[Document(metadata={'标题1': '欢迎来到尚硅谷!'}, page_content='欢迎来到尚硅谷!'),
 Document(metadata={'标题1': '欢迎来到尚硅谷!'}, page_content='尚硅谷是专门培训IT技术方向'),
 Document(metadata={'标题1': '欢迎来到尚硅谷!'; '标题2': '尚硅谷老师简介'},
 page_content='尚硅谷老师简介'),
 Document(metadata={'标题1': '欢迎来到尚硅谷!'; '标题2': '尚硅谷老师简介'},
 page_content='尚硅谷老师拥有多年教学经验，都是从一线互联网下来'),
 Document(metadata={'标题1': '欢迎来到尚硅谷!'; '标题2': '尚硅谷老师简介'; '标题3': '尚硅谷北京校区'},
 page_content='尚硅谷北京校区'),
 Document(metadata={'标题1': '欢迎来到尚硅谷!'; '标题2': '尚硅谷老师简介'; '标题3': '尚硅谷北京校区'},
 page_content='北京校区位于宏福科技园区')]

```

说明：

- 标题下文本内容所属标题的层级信息保存在元数据中。
- 每个分块会自动继承父级标题的上下文，避免信息割裂。

## 类型2：CodeTextSplitter：Split code

CodeTextSplitter是一个 **专为代码文件** 设计的文本分割器（Text Splitter），支持代码的语言包括['cpp', 'go', 'java', 'js', 'php', 'proto', 'python', 'rst', 'ruby', 'rust', 'scala', 'swift', 'markdown', 'latex', 'html', 'sol']。它能够根据编程语言的语法结构（如函数、类、代码块等）智能地拆分代码，保持代码逻辑的完整性。

与递归文本分割器（如RecursiveCharacterTextSplitter）不同，CodeTextSplitter 针对代码的特性进行了优化，**避免在函数或类的中间截断**。

举例1：支持的语言

```
1 pip install langchain-text-splitters
```

```

1 from langchain.text_splitter import Language
2
3
4 # 支持分割语言类型
5 # Full list of supported languages
6 langs = [e.value for e in Language]
7 print(langs)

```

```

['cpp', 'go', 'java', 'kotlin', 'js', 'ts', 'php', 'proto', 'python', 'rst', 'ruby', 'rust', 'scala', 'swift',
'markdown', 'latex', 'html', 'sol', 'csharp', 'cobol', 'c', 'lua', 'perl', 'haskell', 'elixir',
'powershell']

```

举例2:

```

1 # 1.导入相关依赖
2 from langchain.text_splitter import (
3 Language,
4 RecursiveCharacterTextSplitter,
5)
6 from pprint import pprint
7
8
9 # 2.定义要分割的python代码片段
10 PYTHON_CODE = """
11 def hello_world():
12 print("Hello, World!")
13
14 def hello_world1():
15 print("Hello, World1!")
16 """
17
18 # 3.定义递归字符切分器
19 python_splitter = RecursiveCharacterTextSplitter.from_language(
20 language=Language.PYTHON,
21 chunk_size=50,
22 chunk_overlap=0
23)
24
25 # 4.文档切分
26 python_docs = python_splitter.create_documents(texts=[PYTHON_CODE])
27
28 pprint(python_docs)

```

```

[Document(metadata={}, page_content='def hello_world():\n print("Hello, World!")'),
 Document(metadata={}, page_content='def hello_world1():\n print("Hello, World1!")')]

```

**类型3: MarkdownTextSplitter: md数据类型**

因为Markdown格式有特定的语法，一般整体内容由 **h1**、**h2**、**h3** 等多级标题组织，所以 MarkdownHeaderTextSplitter的切分策略就是根据 **标题来分割文本内容**。

举例：

```
1 from langchain.text_splitter import MarkdownTextSplitter
2
3 markdown_text = """
4 # 一级标题\n
5 这是一级标题下的内容\n\n
6 ## 二级标题\n
7 - 二级下列表项1\n
8 - 二级下列表项2\n
9 """
10
11 # 关键步骤：直接修改实例属性
12 splitter = MarkdownTextSplitter(chunk_size=30, chunk_overlap=0)
13 splitter.is_separator_regex = True # 强制将分隔符视为正则表达式
14
15 # 执行分割
16 docs = splitter.create_documents(texts = [markdown_text])
17
18 # print(len(docs))
19
20 for i, doc in enumerate(docs):
21 print(f"\n🔍 分块 {i + 1}:")
22 print(doc.page_content)
```

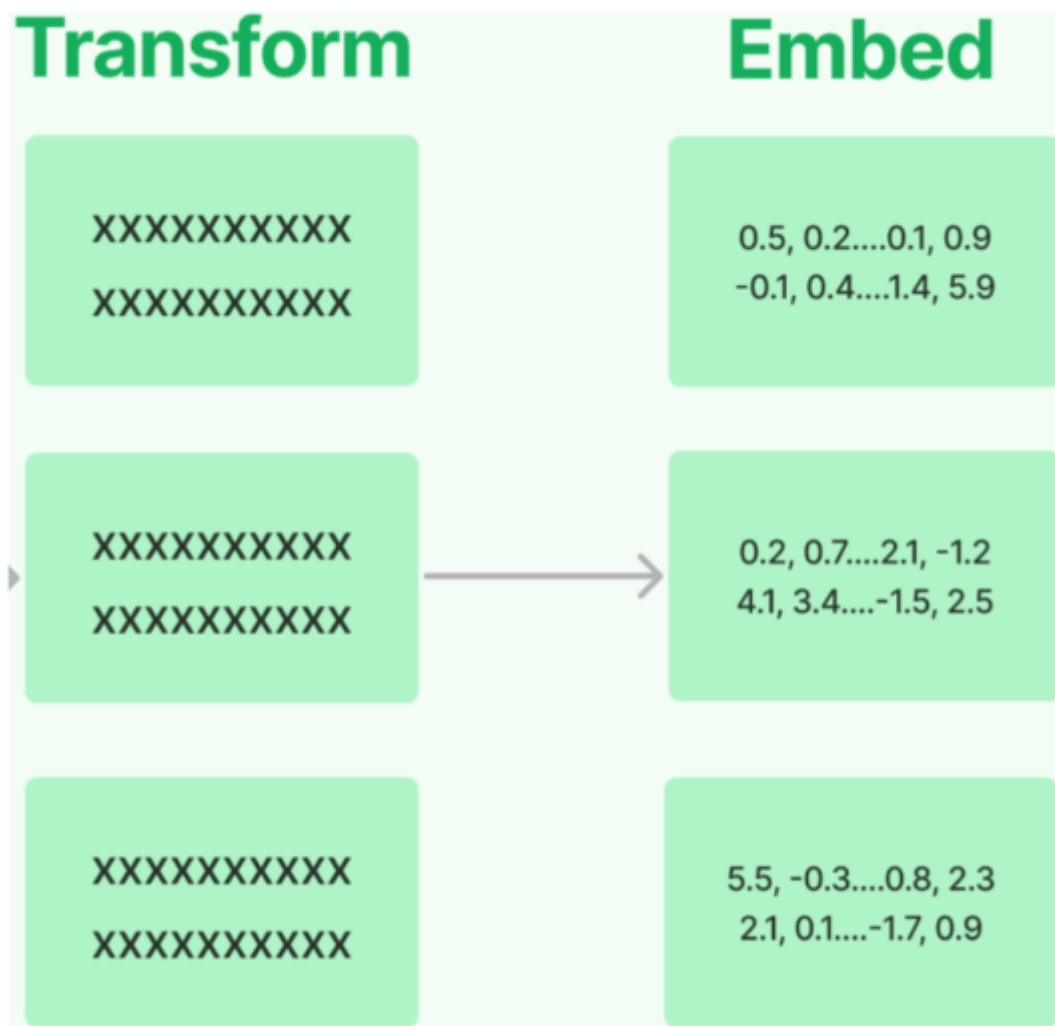
```
1 🔍 分块 1:
2 # 一级标题
3
4 这是一级标题下的内容
5
6 🔍 分块 2:
7 ## 二级标题
8
9 - 二级下列表项1
10
11 - 二级下列表项2
```

## 4、文档嵌入模型 Text Embedding Models

### 4.1 嵌入模型概述

**Text Embedding Models**：文档嵌入模型，提供将文本编码为向量的能力，即 **文档向量化**。**文档写入**和 **用户查询匹配** 前都会先执行文档嵌入编码，即向量化。





- LangChain提供了 **超过25种** 不同的嵌入提供商和方法的集成，从开源到专有API，总有一款适合你。
- Hugging Face等开源社区提供了一些文本向量化模型（例如BGE），效果比闭源且调用API的向量化模型效果好，并且**向量化模型参数量小，在CPU上即可运行**。所以，这里推荐在开发RAG应用的过程中，使用 **开源的文本向量化模型**。此外，开源模型还可以根据应用场景下收集的数据对模型进行微调，提高模型效果。

LangChain中针对向量化模型的封装提供了两种接口，一种针对 **文档的向量化(embed\_documents)**，一种针对 **句子的向量化embed\_query**。

## 4.2 句子的向量化 (embed\_query)

举例：

```
1 from langchain_openai import OpenAIEmbeddings
2 import os
3 import dotenv
4
5 dotenv.load_dotenv()
6
7 os.environ['OPENAI_API_KEY'] = os.getenv("OPENAI_API_KEY")
8 os.environ['OPENAI_BASE_URL'] = os.getenv("OPENAI_BASE_URL")
9
10 # 初始化嵌入模型
11 embeddings_model = OpenAIEmbeddings(model="text-embedding-ada-002")
12 # embeddings_model = OpenAIEmbeddings(model="text-embedding-3-large")
```

```

13
14 # 待嵌入的文本句子
15 text = "What was the name mentioned in the conversation?"
16
17 # 生成一个嵌入向量
18 embedded_query = embeddings_model.embed_query(text = text)
19
20 # 使用embedded_query[:5]来查看前5个元素的值
21 print(embedded_query[:5])
22
23 print(len(embedded_query))

```

```

[0.005329647101461887, -0.0006122003542259336, 0.0389961302280426,
-0.002898985054343939, -0.008904732763767242]
1536

```

## 4.3 文档的向量化 (embed\_documents)

文档的向量化，接收的参数是字符串数组。

举例1：

```

1 from langchain_openai import OpenAIEmbeddings
2 import numpy as np
3 import pandas as pd
4 import os
5 import dotenv
6
7 dotenv.load_dotenv()
8
9 os.environ['OPENAI_API_KEY'] = os.getenv("OPENAI_API_KEY")
10 os.environ['OPENAI_BASE_URL'] = os.getenv("OPENAI_BASE_URL")
11
12 # 初始化嵌入模型
13 embeddings_model = OpenAIEmbeddings(model="text-embedding-ada-002")
14
15 # 待嵌入的文本列表
16 texts = [
17 "Hi there!",
18 "Oh, hello!",
19 "What's your name?",
20 "My friends call me World",
21 "Hello World!"
22]
23
24 # 生成嵌入向量
25 embeddings = embeddings_model.embed_documents(texts)
26
27
28 for i in range(len(texts)):
29 print(f"{texts[i]}:{embeddings[i][:3]}",end= "\n\n")
30

```

Hi there!:[-0.020325319841504097, -0.007096723187714815, -0.022839006036520004]  
Oh, hello!:[0.00445469468832016, -0.014359182678163052, 0.0019080477068200707]  
What's your name?:[-0.00477176159620285, -0.009507440961897373, 0.00713208457455039]  
My friends call me World:[-0.004583988804370165, -0.014502654783427715, 0.010228524915874004]  
Hello World!:[0.002363691572099924, 0.00023463694378733635, -0.00233377143740654]

举例2:

```
1 from dotenv import load_dotenv
2 from langchain_community.document_loaders import CSVLoader
3 from langchain_openai import OpenAIEmbeddings
4
5
6 embeddings_model = OpenAIEmbeddings(
7 model="text-embedding-3-large",
8)
9
10 # 情况1:
11 loader = CSVLoader("./asset/load/03-load.csv", encoding="utf-8")
12 docs = loader.load_and_split()
13
14 # print(len(docs))
15
16 # 存放的是每一个chunk的embedding。
17 embedded_docs = embeddings_model.embed_documents([doc.page_content for doc in docs])
18 print(len(embedded_docs))
19 # 表示的是每一个chunk的embedding的维度
20 print(len(embedded_docs[0]))
21 print(embedded_docs[0][:10])
```

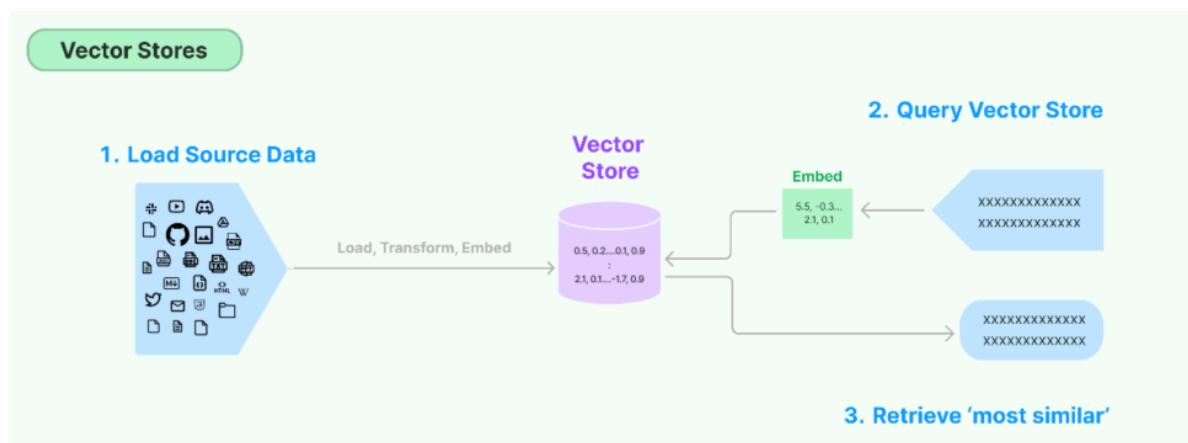
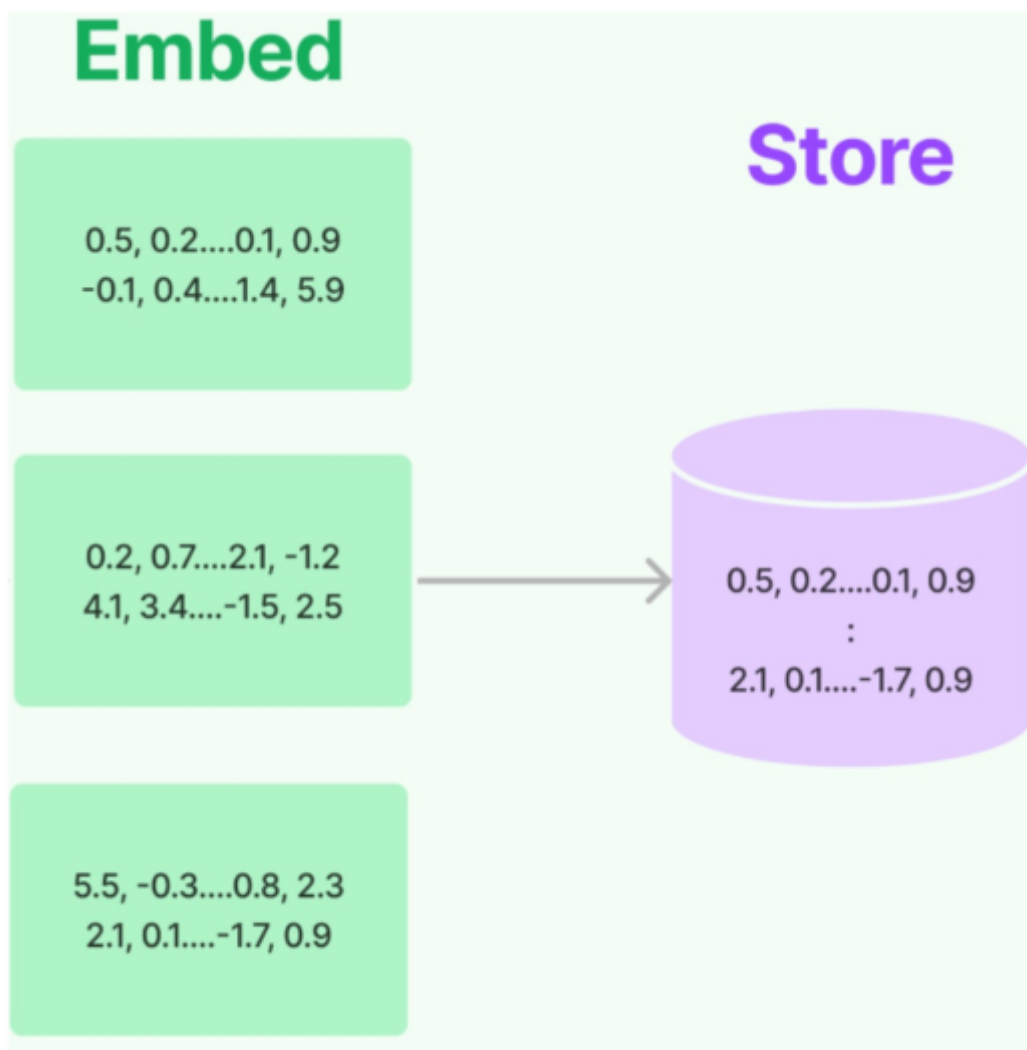
```
4
3072
[0.0011628984939306974, 0.005325784906744957, -0.008948351256549358,
0.03101922944188118, -0.0018112830584868789, 0.011116084642708302,
0.02107088267803192, 0.04843851551413536, 0.0002078620600514114,
-0.00013618897355627269]
```

## 5、向量存储(Vector Stores)

## 5.1 理解向量存储

将文本向量化之后，下一步就是进行向量的存储。这部分包含两块：

- **向量的存储**：将非结构化数据向量化后，完成存储
- **向量的查询**：查询时，嵌入非结构化查询并检索与嵌入查询“最相似”的嵌入向量。即具有相似性检索能力



## 5.2 常用的向量数据库

LangChain提供了超过 50种 不同向量存储（Vector Stores）的集成，从开源的 本地向量存储 到 云托管 的私有向量存储，允许你选择最适合需求的向量存储。

LangChain支持的向量存储参考 `VectorStore` 接口和实现。

# Vector Stores: 向量数据库

- Alibaba Cloud OpenSearch
  - AnalyticDB
  - Annoy
  - Atlas
  - AwaDB
  - Azure Cognitive Search
  - BagelDB
  - Cassandra
  - **Chroma**
  - Clarifai
  - ClickHouse Vector Search
  - Activeloop's Deep Lake
  - Dingo
  - DocArrayHnswSearch
  - DocArrayInMemorySearch
  - ElasticSearch
  - FAISS
  - Hologres
  - LanceDB
  - Marqo
  - MatchingEngine
  - Meilisearch
  - Milvus
- MongoDB Atlas
  - MyScale
  - OpenSearch
  - pg\_embedding
  - **PGVector**
  - **Pinecone**
  - Qdrant
  - Redis
  - Rockset
  - ScaNN
  - SingleStoreDB
  - scikit-learn
  - StarRocks
  - Supabase (Postgres)
  - Tair
  - Tigris
  - Typesense
  - USearch
  - Vectara
  - Weaviate
  - Xata
  - Zilliz

典型的介绍如下：

向量数据库	描述
Chroma	开源、免费的嵌入式数据库
FAISS	Meta出品，开源、免费，Facebook AI相似性搜索服务。（Facebook AI Similarity Search, Facebook AI 相似性搜索库） /fæs/
Milvus	用于存储、索引和管理由深度神经网络和其他ML模型产生的大量嵌入向量的数据库
Pinecone	具有广泛功能的向量数据库
Redis	基于Redis的检索器

## 5.3 向量数据库的理解

假设你是一名摄影师，拍了大量的照片。为了方便管理和查找，你决定将这些 **照片存储** 到一个数据库中。传统的 **关系型数据库**（如 MySQL、PostgreSQL 等）可以帮助你 **存储照片的元数据**，比如拍摄时间、地点、相机型号等。

但是，当你想要根据 **照片的内容（如颜色、纹理、物体等）** 进行搜索时，传统数据库将无法满足你的需求，因为它们通常以数据表的形式存储数据，并使用查询语句进行精确搜索。那么此时，**向量数据库就可以派上用场**。

我们可以构建一个多维的空间使得每张照片特征都存在于这个空间内，并用已有的维度进行表示，比如时间、地点、相机型号、颜色.....**此照片的信息将作为一个点，存储于其中**。以此类推，即可在该空间中构建出无数的点，而后我们将这些点与空间坐标轴的原点相连接，就成为了一条条向量，当这些点变为向量之后，即可利用向量的计算进一步获取更多的信息。当要进行照片的检索时，也会变得更容易更快捷。

**注意：**在向量数据库中进行检索时，检索并 **不是唯一的、精确的**，而是查询和目标向量 **最为相似的一些向量**，具有模糊性。

**延伸思考：**只要对图片、视频、商品等素材进行向量化，就可以实现以图搜图、视频相关推荐、相似宝贝推荐等功能。

## 5.4 代码实现

使用向量数据库组件时需要同时传入包含 **文本块的Document类对象** 以及 **文本向量化组件**，向量数据库组件会自动完成将文本向量化的工作，并写入数据库中。

### 5.4.1 数据的存储

举例1：从TXT文档中加载数据，向量化后存储到Chroma数据库

安装模块：

```
1 pip install chromadb
2
3 pip install langchain-chroma
```

```
1 from langchain_text_splitters import CharacterTextSplitter
2 from langchain_community.vectorstores import Chroma
3 from langchain_community.document_loaders import TextLoader
4 from langchain_openai import OpenAIEmbeddings
5
6 # 举例：将分割后的文本，使用 OpenAI 嵌入模型获取嵌入向量，并存储在 Chroma 中
7
8 # 获取嵌入模型
9 my_embedding = OpenAIEmbeddings(model="text-embedding-ada-002")
10
11 # 创建TextLoader实例，并加载指定的文档
12 loader = TextLoader("./asset/load/09-ai1.txt", encoding='utf-8')
13 documents = loader.load()
14
15 # 创建文本拆分器
16 text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
17 # 拆分文档
18 docs = text_splitter.split_documents(documents)
```

```
19
20 # 存储: 将文档和数据存储到向量数据库中
21 db = Chroma.from_documents(docs, my_embedding)
22
23 # 查询: 使用相似度查找
24 query = "人工智能的核心技术有啥?"
25 docs = db.similarity_search(query)
26 print(docs[0].page_content)
```

**思考: 此时数据存储在哪里呢?**

**注意:** Chroma主要有两种存储模式: **内存模式** 和 **持久化模式**。当使用persist\_directory参数时, 数据会保存到指定目录; 如果没有指定, 则默认使用内存存储。

### 3. 人工智能的核心技术

#### 3.1 机器学习

机器学习是人工智能的核心技术之一, 通过算法使计算机从数据中学习并做出决策。常见的机器学习算法包括监督学习、无监督学习和强化学习。监督学习通过标记数据进行训练, 无监督学习则从未标记数据中寻找模式, 强化学习则通过与环境交互来优化决策。

#### 3.2 深度学习

深度学习是机器学习的一个子领域, 通过多层神经网络进行特征提取和模式识别。深度学习在图像识别、自然语言处理、语音识别等领域取得了显著成果。常见的深度学习模型包括卷积神经网络 (CNN)、循环神经网络 (RNN) 和长短期记忆网络 (LSTM)。

#### 3.3 自然语言处理

自然语言处理 (NLP) 是人工智能的一个重要分支, 致力于使计算机能够理解和生成人类语言。NLP技术广泛应用于机器翻译、情感分析、文本分类等领域。近年来, 基于深度学习的NLP模型 (如BERT、GPT) 在语言理解任务中取得了突破性进展。

#### 3.4 计算机视觉

计算机视觉是人工智能的另一个重要分支, 致力于使计算机能够理解和处理图像和视频。计算机视觉技术广泛应用于图像识别、目标检测、人脸识别等领域。深度学习模型 (如CNN) 在计算机视觉任务中取得了显著成果。

### 4. 人工智能的应用领域

#### 4.1 医疗健康

人工智能在医疗健康领域的应用包括疾病诊断、药物研发、个性化医疗等。通过分析医学影像和患者数据, 人工智能可以帮助医生更准确地诊断疾病, 提高治疗效果。

#### 4.2 金融

人工智能在金融领域的应用包括风险评估、欺诈检测、算法交易等。通过分析市场数据和交易记录, 人工智能可以帮助金融机构做出更明智的决策, 提高运营效率。

#### 4.3 教育

人工智能在教育领域的应用包括个性化学习、智能辅导、自动评分等。通过分析学生的学习数据, 人工智能可以为学生提供个性化的学习建议, 提高学习效果。

#### 4.4 交通

人工智能在交通领域的应用包括自动驾驶、交通管理、智能导航等。通过分析交通数据和路况信息, 人工智能可以帮助优化交通流量, 提高交通安全。

**举例2: 操作csv文档, 并向量化**

```
1 from langchain.text_splitter import CharacterTextSplitter
2 from langchain_community.document_loaders import CSVLoader
3 from langchain_openai import OpenAIEmbeddings
4 from langchain_chroma import Chroma
```



```

5
6 import os
7 import dotenv
8
9 dotenv.load_dotenv()
10 os.environ['OPENAI_API_KEY'] = os.getenv("OPENAI_API_KEY")
11 os.environ['OPENAI_BASE_URL'] = os.getenv("OPENAI_BASE_URL")
12
13 # 获取嵌入模型
14 embeddings = OpenAIEmbeddings(model="text-embedding-ada-002")
15
16 # 加载文档并拆分 (第1次拆分)
17 loader = CSVLoader("./asset/load/03-load.csv", encoding='utf-8')
18 pages = loader.load_and_split()
19 #print(len(pages)) # 4
20
21 # 文本拆分 (第2次拆分)
22 text_splitter = CharacterTextSplitter.from_tiktoken_encoder(chunk_size=500)
23 docs = text_splitter.split_documents(pages)
24
25 # 向量存储
26 db_path = './chroma_db'
27 db = Chroma.from_documents(docs, embeddings, persist_directory=db_path)

```

## 5.4.2 数据的检索

举例：一个包含构建Chroma向量数据库以及向量检索的代码

前置代码：

```

1 # 1.导入相关依赖
2 from langchain_chroma import Chroma
3 from langchain_core.documents import Document
4 from langchain_openai import OpenAIEmbeddings
5
6 # 2.定义文档
7 raw_documents = [
8 Document(
9 page_content="葡萄是一种常见的水果，属于葡萄科葡萄属植物。它的果实呈圆形或椭圆形，颜色有绿色、紫色、红色等多种。葡萄富含维生素C和抗氧化物质，可以直接食用或酿造成葡萄酒。",
10 metadata={"source": "水果", "type": "植物"}
11),
12 Document(
13 page_content="白菜是十字花科蔬菜，原产于中国北方。它的叶片层层包裹形成紧密的球状，口感清脆微甜。白菜富含膳食纤维和维生素K，常用于制作泡菜、炒菜或煮汤。",
14 metadata={"source": "蔬菜", "type": "植物"}
15),
16 Document(
17 page_content="狗是人类最早驯化的动物之一，属于犬科。它们具有高度社会性，能理解人类情绪，常被用作宠物、导盲犬或警犬。不同品种的狗在体型、毛色和性格上有很大差异。",
18 metadata={"source": "动物", "type": "哺乳动物"}
19),

```

```

20 Document(
21 page_content= "猫是小型肉食性哺乳动物，性格独立但也能与人类建立亲密关系。它们夜视能力
 极强，擅长捕猎老鼠。家猫的品种包括波斯猫、暹罗猫等，毛色和花纹多样。 ";
22 metadata={ "source": "动物", "type": "哺乳动物"
23 },
24 Document(
25 page_content= "人类是地球上最具智慧的生物，属于灵长目人科。现代人类（智人）拥有高度发
 达的大脑，创造了语言、工具和文明。人类的平均寿命约70-80年，分布在全球各地。 ";
26 metadata={ "source": "生物", "type": "灵长类"
27 },
28 Document(
29 page_content= "太阳是太阳系的中心恒星，直径约139万公里，主要由氢和氦组成。它通过核聚
 变反应产生能量，为地球提供光和热。太阳活动周期约为11年，会影响地球气候。 ";
30 metadata={ "source": "天文", "type": "恒星"
31 },
32 Document(
33 page_content= "长城是中国古代的军事防御工程，总长度超过2万公里。它始建于春秋战国时
 期，秦朝连接各段，明朝大规模重修。长城是世界文化遗产和人类建筑奇迹。 ";
34 metadata={ "source": "历史", "type": "建筑"
35 },
36 Document(
37 page_content= "量子力学是研究微观粒子运动规律的物理学分支。它提出了波粒二象性、测不准
 原理等概念，彻底改变了人类对物质世界的认知。量子计算机正是基于这一理论发展而来。 ";
38 metadata={ "source": "物理", "type": "科学"
39 },
40 Document(
41 page_content= "《红楼梦》是中国古典文学四大名著之一，作者曹雪芹。小说以贾、史、王、薛
 四大家族的兴衰为背景，描绘了贾宝玉与林黛玉的爱情悲剧，反映了封建社会的种种矛盾。 ";
42 metadata={ "source": "文学", "type": "小说"
43 },
44 Document(
45 page_content= "新冠病毒 (SARS-CoV-2) 是一种可引起呼吸道疾病的冠状病毒。它通过飞沫传
 播，主要症状包括发热、咳嗽、乏力。疫苗和戴口罩是有效的预防措施。 ";
46 metadata={ "source": "医学", "type": "病毒"
47)
48]
49 # 3. 创建嵌入模型
50 embedding = OpenAIEmbeddings(model= "text-embedding-ada-002")
51
52 # 4. 创建向量数据库
53 db = Chroma.from_documents(
54 documents=raw_documents,
55 embedding=embedding,
56 persist_directory= "./asset/chroma-3",
57)

```

## ① 相似性检索 (similarity\_search)

接收字符串作为参数：

```

1 # 5. 检索示例 (返回前3个最相关结果)
2 query = "哺乳动物"
3 docs = db.similarity_search(query, k=3) # k=3表示返回3个最相关文档
4 print(f"查询: '{query}' 的结果:")
5 for i, doc in enumerate(docs, 1):
6 print(f"\n结果 {i}:")
7 print(f"内容: {doc.page_content}")
8 print(f"元数据: {doc.metadata}")

```

查询: '哺乳动物' 的结果:

结果 1:

内容: 猫是小型肉食性哺乳动物，性格独立但也能与人类建立亲密关系。它们夜视能力极强，擅长捕猎老鼠。家猫的品种包括波斯猫、暹罗猫等，毛色和花纹多样。

元数据: {'source': '动物', 'type': '哺乳动物'}

结果 2:

内容: 狗是人类最早驯化的动物之一，属于犬科。它们具有高度社会性，能理解人类情绪，常被用作宠物、导盲犬或警犬。不同品种的狗在体型、毛色和性格上有很大差异。

元数据: {'source': '动物', 'type': '哺乳动物'}

结果 3:

内容: 人类是地球上最具智慧的生物，属于灵长目人科。现代人类（智人）拥有高度发达的大脑，创造了语言、工具和文明。人类的平均寿命约70-80年，分布在全球各地。

元数据: {'source': '生物', 'type': '灵长类'}

## ② 支持直接对问题向量查询 (similarity\_search\_by\_vector)

搜索与给定嵌入向量相似的文档，它接受嵌入向量作为参数，而不是字符串。

```

1 query = "哺乳动物"
2 embedding_vector = embedding.embed_query(query)
3
4 docs = db.similarity_search_by_vector(embedding_vector, k=3)
5
6 print(f"查询: '{query}' 的结果:")
7 for i, doc in enumerate(docs, 1):
8 print(f"\n结果 {i}:")
9 print(f"内容: {doc.page_content}")
10 print(f"元数据: {doc.metadata}")

```

查询: '哺乳动物' 的结果:

结果 1:

内容: 猫是小型肉食性哺乳动物，性格独立但也能与人类建立亲密关系。它们夜视能力极强，擅长捕猎老鼠。家猫的品种包括波斯猫、暹罗猫等，毛色和花纹多样。

元数据: {'source': '动物', 'type': '哺乳动物'}

结果 2:

内容: 狗是人类最早驯化的动物之一, 属于犬科。它们具有高度社会性, 能理解人类情绪, 常被用作宠物、导盲犬或警犬。不同品种的狗在体型、毛色和性格上有很大差异。

元数据: {'source': '动物', 'type': '哺乳动物'}

结果 3:

内容: 人类是地球上最具智慧的生物, 属于灵长目人科。现代人类(智人)拥有高度发达的大脑, 创造了语言、工具和文明。人类的平均寿命约70-80年, 分布在全球各地。

元数据: {'source': '生物', 'type': '灵长类'}

### ③ 相似性检索, 支持过滤元数据 (filter)

```
1 query = "哺乳动物"
2
3 docs = db.similarity_search(
4 query=query,
5 k=3,
6 filter={"source": "动物"})
7
8 for i, doc in enumerate(docs, 1):
9 print(f"\n结果 {i}:")
10 print(f"内容: {doc.page_content}")
11 print(f"元数据: {doc.metadata}")
12
```

结果 1:

内容: 猫是小型肉食性哺乳动物, 性格独立但也能与人类建立亲密关系。它们夜视能力极强, 擅长捕猎老鼠。家猫的品种包括波斯猫、暹罗猫等, 毛色和花纹多样。

元数据: {'source': '动物', 'type': '哺乳动物'}

结果 2:

内容: 狗是人类最早驯化的动物之一, 属于犬科。它们具有高度社会性, 能理解人类情绪, 常被用作宠物、导盲犬或警犬。不同品种的狗在体型、毛色和性格上有很大差异。

元数据: {'type': '哺乳动物', 'source': '动物'}

### ④ 通过L2距离分数进行搜索 (similarity\_search\_with\_score)

说明: 分数值越小, 检索到的文档越和问题相似。分值取值范围: [0, 正无穷]

```
1 docs = db.similarity_search_with_score(
2 "量子力学是什么?"
3)
4 for doc, score in docs:
5 print(f" [L2距离得分={score:.3f}] {doc.page_content} [{doc.metadata}]")
```

[L2距离得分=0.182] 量子力学是研究微观粒子运动规律的物理学分支。它提出了波粒二象性、测不准原理等概念，彻底改变了人类对物质世界的认知。量子计算机正是基于这一理论发展而来。

[{'type': '科学', 'source': '物理'}]

[L2距离得分=0.447] 太阳是太阳系的中心恒星，直径约139万公里，主要由氢和氦组成。它通过核聚变反应产生能量，为地球提供光和热。太阳活动周期约为11年，会影响地球气候。 [{'source': '天文', 'type': '恒星'}]

[L2距离得分=0.463] 人类是地球上最具智慧的生物，属于灵长目人科。现代人类（智人）拥有高度发达的大脑，创造了语言、工具和文明。人类的平均寿命约70-80年，分布在全球各地。

[{'type': '灵长类', 'source': '生物'}]

[L2距离得分=0.488] 新冠病毒（SARS-CoV-2）是一种可引起呼吸道疾病的冠状病毒。它通过飞沫传播，主要症状包括发热、咳嗽、乏力。疫苗和戴口罩是有效的预防措施。 [{'source': '医学', 'type': '病毒'}]

### ⑤ 通过余弦相似度分数进行搜索（\_similarity\_search\_with\_relevance\_scores）

说明：分数值越接近1（上限），检索到的文档越和问题相似。

```
1 docs = db.similarity_search_with_relevance_scores(
2 "量子力学是什么?"
3)
4 for doc, score in docs:
5 print(f"* [余弦相似度得分={score:.3f}] {doc.page_content} [{doc.metadata}]")
```

- 1 \* [余弦相似度得分=0.871] 量子力学是研究微观粒子运动规律的物理学分支。它提出了波粒二象性、测不准原理等概念，彻底改变了人类对物质世界的认知。量子计算机正是基于这一理论发展而来。 [{'type': '科学', 'source': '物理'}]
- 2 \* [余弦相似度得分=0.684] 太阳是太阳系的中心恒星，直径约139万公里，主要由氢和氦组成。它通过核聚变反应产生能量，为地球提供光和热。太阳活动周期约为11年，会影响地球气候。 [{'source': '天文', 'type': '恒星'}]
- 3 \* [余弦相似度得分=0.672] 人类是地球上最具智慧的生物，属于灵长目人科。现代人类（智人）拥有高度发达的大脑，创造了语言、工具和文明。人类的平均寿命约70-80年，分布在全球各地。 [{'source': '生物', 'type': '灵长类'}]
- 4 \* [余弦相似度得分=0.655] 新冠病毒（SARS-CoV-2）是一种可引起呼吸道疾病的冠状病毒。它通过飞沫传播，主要症状包括发热、咳嗽、乏力。疫苗和戴口罩是有效的预防措施。 [{'source': '医学', 'type': '病毒'}]

### ⑥ MMR（最大边际相关性，max\_marginal\_relevance\_search）

MMR 是一种平衡 **相关性** 和 **多样性** 的检索策略，避免返回高度相似的冗余结果。

```
1 docs = db.max_marginal_relevance_search(
2 query="量子力学是什么",
3 lambda_mult=0.8, # 侧重相似性
4)
5
6 print("🔍 关于【量子力学是什么】的搜索结果: ")
7 print("=" * 50)
8 for i, doc in enumerate(docs):
9 print(f"\n📄 结果 {i+1}:")
10 print(f"📄 内容: {doc.page_content}")
11 print(f"🏷️ 标签: {' '.join(f'{k}={v}' for k, v in doc.metadata.items())}")
```

参数说明: `lambda_mult` 参数值介于 0 到 1 之间, 用于确定结果之间的多样性程度, 其中 0 对应最大多样性, 1 对应最小多样性。默认值为 0.5。

## 6、检索器(召回器) Retrievers

### 6.1 介绍

从“向量存储组件”的代码实现5.4.2中可以看到, 向量数据库本身已经包含了实现召回功能的函数方法 ( `similarity_search` )。该函数通过计算原始查询向量与数据库中存储向量之间的相似度来实现召回。

LangChain还提供了 **更加复杂的召回策略**, 这些策略被集成在Retrievers (检索器或召回器) 组件中。

Retrievers (检索器) 是一种用于从大量文档中检索与给定查询相关的文档或信息片段的工具。检索器 **不需要存储文档**, 只需要 **返回 (或检索) 文档** 即可。

召回器组件类名	功能
MultiQueryRetriever	用 LLM 将用户查询改写成多个查询
ContextualCompressionRetriever	用 LLM 对召回文本进行压缩或过滤
EnsembleRetriever	集成多个召回器
LongContextReorder	将召回内容进行重新排序
MultiVectorRetriever	为每个候选文档保存多个向量表示
ParentDocumentRetriever	小块文本向量当作索引, 返回大块文本
SelfQueryRetriever	将自然语言查询转化为结构化查询
TimeWeightedVectorStoreRetriever	召回文本时考虑时效性因素

#### Retrievers 的执行步骤:

- 步骤1: 将输入查询转换为向量表示。
- 步骤2: 在向量存储中搜索与查询向量最相似的文档向量 (通常使用余弦相似度或欧几里得距离等度量方法)。
- 步骤3: 返回与查询最相关的文档或文本片段, 以及它们的相似度得分。

## 6.2 代码实现

Retriever 一般和 VectorStore 配套实现，通过 `as_retriever()` 方法获取。

举例：

```
1 # 1.导入相关依赖
2 from langchain_community.document_loaders import TextLoader
3 from langchain_community.vectorstores import FAISS
4 from langchain_openai import OpenAIEmbeddings
5 from langchain_text_splitters import CharacterTextSplitter
6 import os
7 import dotenv
8 dotenv.load_dotenv()
9
10
11 # 2.定义文档加载器
12 loader = TextLoader(file_path='./asset/load/09-ai1.txt',encoding="utf-8")
13
14 # 3.加载文档
15 documents = loader.load()
16
17 # 4.定义文本分割器
18 text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
19
20 # 5.切割文档
21 docs = text_splitter.split_documents(documents)
22
23 # 6.定义嵌入模型
24 os.environ['OPENAI_API_KEY'] = os.getenv("OPENAI_API_KEY1")
25 os.environ['OPENAI_BASE_URL'] = os.getenv("OPENAI_BASE_URL")
26 embeddings = OpenAIEmbeddings(
27 model="text-embedding-3-large"
28)
29
```

```
1 # 7.将文档存储到向量数据库中
2 db = FAISS.from_documents(docs, embeddings)
3
4 # 8.从向量数据库中得到检索器
5 retriever = db.as_retriever()
6
7 # 9.使用检索器检索
8 docs = retriever.invoke("深度学习是什么? ")
9
10 print(len(docs))
11
12 # 10.得到结果
13 for doc in docs:
14 print(f"★{doc}")
```



## 摘要

人工智能（Artificial Intelligence, AI）作为计算机科学的一个重要分支，近年来取得了突飞猛进的发展。本文综述了人工智能的发展历程、核心技术、应用领域以及未来发展趋势。通过对人工智能的定义、历史背景、主要技术（如机器学习、深度学习、自然语言处理等）的详细介绍，探讨了人工智能在医疗、金融、教育、交通等领域的应用，并分析了人工智能发展过程中面临的挑战与机遇。最后，本文对人工智能的未来发展进行了展望，提出了可能的突破方向。

## 1. 引言

人工智能是指通过计算机程序模拟人类智能的一门科学。自20世纪50年代诞生以来，人工智能经历了多次起伏，近年来随着计算能力的提升和大数据的普及，人工智能技术取得了显著的进展。人工智能的应用已经渗透到日常生活的方方面面，从智能手机的语音助手到自动驾驶汽车，从医疗诊断到金融分析，人工智能正在改变着人类社会的运行方式。

## 2. 人工智能的发展历程

### 2.1 早期发展

人工智能的概念最早可以追溯到20世纪50年代。1956年，达特茅斯会议（Dartmouth Conference）被认为是人工智能研究的正式开端。在随后的几十年里，人工智能研究经历了多次高潮与低谷。早期的研究主要集中在符号逻辑和专家系统上，但由于计算能力的限制和算法的不足，进展缓慢。

### 2.2 机器学习的兴起

20世纪90年代，随着统计学习方法的引入，机器学习逐渐成为人工智能研究的主流。支持向量机（SVM）、决策树、随机森林等算法在分类和回归任务中取得了良好的效果。这一时期，机器学习开始应用于数据挖掘、模式识别等领域。

### 2.3 深度学习的突破

2012年，深度学习在图像识别领域取得了突破性进展，标志着人工智能进入了一个新的阶段。深度学习通过多层神经网络模拟人脑的工作方式，能够自动提取特征并进行复杂的模式识别。卷积神经网络（CNN）、循环神经网络（RNN）和长短期记忆网络（LSTM）等深度学习模型在图像处理、自然语言处理、语音识别等领域取得了显著成果。' metadata={source: './asset/load/09-ai1.txt'}

## ★page\_content='3. 人工智能的核心技术

### 3.1 机器学习

机器学习是人工智能的核心技术之一，通过算法使计算机从数据中学习并做出决策。常见的机器学习算法包括监督学习、无监督学习和强化学习。监督学习通过标记数据进行训练，无监督学习则从未标记数据中寻找模式，强化学习则通过与环境交互来优化决策。

### 3.2 深度学习

深度学习是机器学习的一个子领域，通过多层神经网络进行特征提取和模式识别。深度学习在图像识别、自然语言处理、语音识别等领域取得了显著成果。常见的深度学习模型包括卷积神经网络（CNN）、循环神经网络（RNN）和长短期记忆网络（LSTM）。

### 3.3 自然语言处理

自然语言处理（NLP）是人工智能的一个重要分支，致力于使计算机能够理解和生成人类语言。NLP技术广泛应用于机器翻译、情感分析、文本分类等领域。近年来，基于深度学习的NLP模型（如BERT、GPT）在语言理解任务中取得了突破性进展。

### 3.4 计算机视觉

计算机视觉是人工智能的另一个重要分支，致力于使计算机能够理解和处理图像和视频。计算机视觉技术广泛应用于图像识别、目标检测、人脸识别等领域。深度学习模型（如CNN）在计算机视觉任务中取得了显著成果。

## 3. 人工智能的应用领域

### 4.1 医疗健康

人工智能在医疗健康领域的应用包括疾病诊断、药物研发、个性化医疗等。通过分析医学影像和患者数据，人工智能可以帮助医生更准确地诊断疾病，提高治疗效果。

#### 4.2 金融

人工智能在金融领域的应用包括风险评估、欺诈检测、算法交易等。通过分析市场数据和交易记录，人工智能可以帮助金融机构做出更明智的决策，提高运营效率。

#### 4.3 教育

人工智能在教育领域的应用包括个性化学习、智能辅导、自动评分等。通过分析学生的学习数据，人工智能可以为学生提供个性化的学习建议，提高学习效果。

#### 4.4 交通

人工智能在交通领域的应用包括自动驾驶、交通管理、智能导航等。通过分析交通数据和路况信息，人工智能可以帮助优化交通流量，提高交通安全。' metadata={'source':

'./asset/load/09-ai1.txt'}

★page\_content='5. 人工智能的挑战与机遇

##### 5.1 挑战

人工智能发展过程中面临的主要挑战包括数据隐私、算法偏见、安全性问题等。数据隐私问题涉及到个人数据的收集和使用，算法偏见问题则涉及到算法的公平性和透明度，安全性问题则涉及到人工智能系统的可靠性和稳定性。

##### 5.2 机遇

尽管面临挑战，人工智能的发展也带来了巨大的机遇。人工智能技术的进步将推动各行各业的创新，提高生产效率，改善生活质量。未来，人工智能有望在更多领域取得突破，为人类社会带来更多的便利和福祉。

#### 4. 未来展望

##### 6.1 技术突破

未来，人工智能技术有望在以下几个方面取得突破：一是算法的优化和创新，提高模型的效率和准确性；二是计算能力的提升，支持更复杂的模型和更大规模的数据处理；三是跨学科研究的深入，推动人工智能与其他领域的融合。

##### 6.2 应用拓展

随着技术的进步，人工智能的应用领域将进一步拓展。未来，人工智能有望在更多领域发挥重要作用，如环境保护、能源管理、智能制造等。人工智能将成为推动社会进步的重要力量。

#### 5. 结论

人工智能作为一门快速发展的科学，正在改变着人类社会的运行方式。通过不断的技术创新和应用拓展，人工智能将为人类社会带来更多的便利和福祉。然而，人工智能的发展也面临着诸多挑战，需要社会各界共同努力，推动人工智能的健康发展。' metadata={'source':

'./asset/load/09-ai1.txt'}

## 6.3 使用相关检索策略

前置代码：

```
1 # 1.导入相关依赖
2 from langchain_community.vectorstores import FAISS
3 from langchain_openai import OpenAIEmbeddings
4 from langchain_core.documents import Document
5
6 # 2.定义文档
7 document_1 = Document(
8 page_content= "经济复苏: 美国经济正在从疫情中强劲复苏, 失业率降至历史低点。! ",
9)
10 document_2 = Document(
11 page_content= "基础设施: 政府将投资1万亿美元用于修复道路、桥梁和宽带网络。 ",
12)
13 document_3 = Document(
```

```

14 page_content= "气候变化: 承诺到2030年将温室气体排放量减少50%。";
15)
16 document_4 = Document(
17 page_content= "医疗保健: 降低处方药价格, 扩大医疗保险覆盖范围。";
18)
19 document_5 = Document(
20 page_content= "教育: 提供免费的社区大学教育。。";
21)
22 document_6 = Document(
23 page_content= "科技: 增加对半导体产业的投资以减少对外国供应链的依赖。。";
24)
25 document_7 = Document(
26 page_content= "外交政策: 继续支持乌克兰对抗俄罗斯的侵略。";
27)
28 document_8 = Document(
29 page_content= "枪支管制: 呼吁国会通过更严格的枪支管制法律。";
30)
31 document_9 = Document(
32 page_content= "移民改革: 提出全面的移民改革方案。";
33)
34 document_10 = Document(
35 page_content= "社会正义: 承诺解决系统性种族歧视问题。";
36)
37 documents = [
38 document_1,
39 document_2,
40 document_3,
41 document_4,
42 document_5,
43 document_6,
44 document_7,
45 document_8,
46 document_9,
47 document_10,
48]
49
50 # 3.创建向量存储
51 embeddings = OpenAIEmbeddings(
52 model= "text-embedding-3-large"
53)
54
55 # 4.将文档向量化, 添加到向量数据库索引中, 得到向量数据库对象
56 db = FAISS.from_documents(documents, embeddings)

```

## ① 默认检索器使用相似性搜索

```

1 # 获取检索器
2 retriever = db.as_retriever(search_kwargs={ "k": 4}) #这里设置返回的文档数
3
4 docs = retriever.invoke("经济政策")
5 for i, doc in enumerate(docs):
6 print(f"\n结果 {i+ 1}:\n{doc.page_content}\n")
7

```

结果 1:  
经济复苏: 美国经济正在从疫情中强劲复苏, 失业率降至历史低点。!

结果 2:  
科技: 增加对半导体产业的投资以减少对外国供应链的依赖。。

结果 3:  
外交政策: 继续支持乌克兰对抗俄罗斯的侵略。

结果 4:  
基础设施: 政府将投资1万亿美元用于修复道路、桥梁和宽带网络。

## ② 分数阈值查询

只有相似度超过这个值才会召回

```
1 retriever = db.as_retriever(
2 search_type= "similarity_score_threshold",
3 search_kwargs={ "score_threshold": 0.1}
4)
5 docs = retriever.invoke("经济政策")
6
7 for doc in docs:
8 print(f"🔗 内容: {doc.page_content}")
```

```
1 🔗 内容: 经济复苏: 美国经济正在从疫情中强劲复苏, 失业率降至历史低点。!
```

注意只会返回满足阈值分数的文档, 不会获取文档的得分。如果想查询文档的得分是否满足阈值, 可以使用向量数据库的 `similarity_search_with_relevance_scores` 查看 (在5.4.2 情况5中讲过)。

```
1 docs_with_scores = db.similarity_search_with_relevance_scores("经济政策")
2
3 for doc, score in docs_with_scores:
4 print(f"\n相似度分数: {score:.4f}")
5 print(f"🔗 内容: {doc.page_content}")
```

```
1 相似度分数: 0.1015
2 🔗 内容: 经济复苏: 美国经济正在从疫情中强劲复苏, 失业率降至历史低点。!
3
4 相似度分数: 0.0482
5 🔗 内容: 科技: 增加对半导体产业的投资以减少对外国供应链的依赖。。
6
7 相似度分数: 0.0451
8 🔗 内容: 外交政策: 继续支持乌克兰对抗俄罗斯的侵略。
9
10 相似度分数: 0.0274
11 🔗 内容: 基础设施: 政府将投资1万亿美元用于修复道路、桥梁和宽带网络。
```

### ③ MMR搜索

```
1 retriever = db.as_retriever(
2 search_type="mmr",
3 # search_kwargs={"fetch_k":2}
4)
5
6 docs = retriever.invoke("经济政策")
7
8 print(len(docs))
9
10 for doc in docs:
11 print(f"🔗 内容: {doc.page_content}")
```

```
1 4
2
3 🔗 内容: 经济复苏: 美国经济正在从疫情中强劲复苏, 失业率降至历史低点。!
4 🔗 内容: 外交政策: 继续支持乌克兰对抗俄罗斯的侵略。
5 🔗 内容: 基础设施: 政府将投资1万亿美元用于修复道路、桥梁和宽带网络。
6 🔗 内容: 社会正义: 承诺解决系统性种族歧视问题。
```

## 6.4 结合LLM

举例1: 通过FAISS构建一个可搜索的向量索引数据库, 并结合RAG技术让LLM去回答问题。

**情况1: 不用RAG给LLM灌输上下文数据**

```
1 from langchain_openai import ChatOpenAI
2 import os
3 import dotenv
4 dotenv.load_dotenv()
5
6 os.environ['OPENAI_API_KEY'] = os.getenv("OPENAI_API_KEY1")
7 os.environ['OPENAI_BASE_URL'] = os.getenv("OPENAI_BASE_URL")
8
9 # 创建大模型实例
10 llm = ChatOpenAI(model="gpt-4o-mini")
11
12 # 调用
13 response = llm.invoke("北京有什么著名的建筑?")
14 print(response.content)
```

```
1 北京有许多著名的建筑, 其中包括:
2
3 1. 故宫: 中国明清两代的皇家宫殿, 也被称为紫禁城, 是世界上最大、保存最完整的古代宫殿建
 筑群之一。
4
5 2. 天安门: 坐落在故宫的南端, 是古代皇城的正门, 也是现代中国政权和国家形象的象征。
```

- 6
- 7 3. 鸟巢：奥林匹克体育中心主体育场，是2008年北京奥运会和残奥会的主要比赛场馆，因其外形像一个鸟巢而得名。
- 8
- 9 4. 雍和宫：北京最大的藏传佛教寺庙之一，也是中国规模最大的藏传佛教寺庙。
- 10
- 11 5. 颐和园：中国古典园林建筑的代表之一，为清代皇家园林，被誉为“皇家园林博物馆”。
- 12
- 13 6. 北京大学：中国最著名的高等学府之一，拥有悠久的历史 and 优秀的师资力量。
- 14
- 15 7. 中央电视台总部大楼（CCTV大楼）：由荷兰建筑师雷姆·库哈设计，是一座反映现代建筑风格的标志性建筑。

## 情况2：使用RAG给LLM灌输上下文数据

```
1 pip install faiss-cpu
```

```
1 # 1. 导入所有需要的包
2 from langchain.prompts import PromptTemplate
3 from langchain_openai import ChatOpenAI, OpenAIEmbeddings
4 from langchain_community.document_loaders import TextLoader
5 from langchain.text_splitter import CharacterTextSplitter
6 from langchain_community.vectorstores import FAISS
7 import os
8 import dotenv
9
10 dotenv.load_dotenv()
11
12 # 2. 创建自定义提示词模板
13 prompt_template = """请使用以下提供的文本内容来回答问题。仅使用提供的文本信息，如果文本中
14 没有相关信息，请回答"抱歉，提供的文本中没有这个信息"。
15
16 文本内容:
17 {context}
18
19 问题: {question}
20
21 回答:
22 """
23
24 prompt = PromptTemplate.from_template(prompt_template)
25
26 # 3. 初始化模型
27 os.environ['OPENAI_API_KEY'] = os.getenv("OPENAI_API_KEY1")
28 os.environ['OPENAI_BASE_URL'] = os.getenv("OPENAI_BASE_URL")
29 llm = ChatOpenAI(
30 model="gpt-4o-mini",
31 temperature=0
32)
33
34 embedding_model = OpenAIEmbeddings(model="text-embedding-3-large")
```

```

35
36 # 4. 加载文档
37 loader = TextLoader("./asset/load/10-test_doc.txt", encoding='utf-8')
38 documents = loader.load()
39
40 # 5. 分割文档
41 text_splitter = CharacterTextSplitter(
42 chunk_size=1000,
43 chunk_overlap=100,
44)
45 texts = text_splitter.split_documents(documents)
46
47 #print(f"文档个数:{len(texts)}")
48
49 # 6. 创建向量存储
50 vectorstore = FAISS.from_documents(
51 documents=texts,
52 embedding=embedding_model
53)
54
55 # 7. 获取检索器
56 retriever = vectorstore.as_retriever()
57
58 docs = retriever.invoke("北京有什么著名的建筑? ")
59
60 # 8. 创建Runnable链
61 chain = prompt | llm
62
63 # 9. 提问
64 result = chain.invoke(input={"question": "北京有什么著名的建筑? ", "context": docs})
65 print("\n回答:", result.content)

```

1. 文档个数:1

回答: 北京有以下著名的建筑:

1. 故宫 - 明清两代的皇家宫殿，世界上现存规模最大、保存最完整的木质结构古建筑群之一。
2. 天安门 - 北京的标志性建筑之一，天安门广场是世界上最大的城市广场。
3. 颐和园 - 清朝时期的皇家园林，融合了江南园林的设计风格。
4. 天坛 - 明清两代皇帝祭天、祈谷的场所，具有深厚的文化内涵。
5. 长城（八达岭段） - 最著名的北京段，被誉为"世界第八大奇迹"。
6. 国家体育场（鸟巢） - 2008年奥运会主体育场，以独特的钢结构设计著称。
7. 中央电视台总部大楼 - 现代北京最具争议和识别度的建筑之一。
8. 国家大剧院 - 因其蛋壳造型被称为"巨蛋"，是世界最大的穹顶建筑之一。
9. 北京大兴国际机场 - 超大型国际航空枢纽，被誉为"新世界七大奇迹"之一。
10. 鼓楼和钟楼 - 古代中国的计时中心，展现了古代的计时智慧。

举例2: 使用Chroma数据库（与举例1类似）



## 阶段1：文档的切分

```
1 ## 1. 文档加载
2 from langchain_community.document_loaders import UnstructuredMarkdownLoader
3 from langchain.text_splitter import MarkdownTextSplitter
4
5 markdown_path = "asset/load/11-langchain.md"
6
7 # 2.定义UnstructuredMarkdownLoader对象
8 loader = UnstructuredMarkdownLoader(markdown_path)
9
10 # 3.加载
11 data = loader.load()
12
13 splitter = MarkdownTextSplitter(chunk_size=1000, chunk_overlap=100)
14
15 # 4.执行分割
16 documents = splitter.split_documents(data)
17 print(len(documents))
18 for i, doc in enumerate(documents):
19 print(f"\n👁 分块 {i + 1}:")
20 print(doc.page_content)
```

## 阶段2：向量存储与检索

```
1 from langchain_openai import OpenAIEmbeddings
2 from langchain_community.vectorstores import Chroma
3
4 # 5. 获取嵌入模型
5 import os
6 import dotenv
7 dotenv.load_dotenv()
8
9 os.environ['OPENAI_API_KEY'] = os.getenv("OPENAI_API_KEY")
10 os.environ['OPENAI_BASE_URL'] = os.getenv("OPENAI_BASE_URL")
11 embeddings = OpenAIEmbeddings()
12
13 # 6. 向量数据存储 (默认存储到内存中)
14 db = Chroma.from_documents(documents, embeddings)
15
16 # 7. 向量检索
17 retriever = db.as_retriever()
18 docs = retriever.invoke("what is Chat Models?")
19 for i, doc in enumerate(docs):
20 print(f"\n👁 分块 {i + 1}:")
21 print(doc.page_content)
22
```

## 阶段3：

```
1 from langchain_openai import ChatOpenAI
2 from langchain.prompts import PromptTemplate
```

```

3
4 llm = ChatOpenAI(model="gpt-4o-mini")
5
6 # 8.定义提示词模版
7 template = """
8 你是一个问答机器人。
9 你的任务是根据下述给定的已知信息回答用户问题。
10 确保你的回复完全依据下述已知信息。不要编造答案。
11 如果下述已知信息不足以回答用户的问题，请直接回复"我无法回答您的问题"。
12
13 已知信息:
14 {context}
15
16 用户问:
17 {question}
18
19 请用中文回答用户问题。
20 """
21 # 7.得到提示词模版对象
22 prompt_template = PromptTemplate.from_template(template=template)
23
24 # 8.得到提示词对象
25 prompt = prompt_template.invoke({"question": "what is Chat Models?", "context": docs})
26
27 ## 9. 调用LLM
28 response = llm.invoke(prompt)
29 print(response.content)

```

- 1 聊天模型（Chat Models）是新型的语言模型，它接收消息并输出消息。请查看特定提供者的支持集成以了解如何开始使用聊天模型。

## 7、项目：智能对话助手

### 7.1 需求分析

我们将构建一个可以与多种不同工具进行交互的Agent：一个是本地数据库，另一个是搜索引擎。你能够向该Agent提问，观察它调用工具，并与它进行对话。

**涉及的功能：**

- 使用 **语言模型**，特别是它们的工具调用能力
- 创建 **检索器** 以向我们的Agent公开特定信息
- 使用 **搜索工具** 在线查找信息
- 提供 **聊天历史**，允许聊天机器人 **“记住”不同id** 过去的交互，并在回答后续问题时考虑它们。

### 7.2 代码实现

## 7.2.1 定义工具

```
1 import os
2 from langchain_community.tools.tavily_search import TavilySearchResults
3
4 # 定义 AVILY_KEY 密钥
5 os.environ["TAVILY_API_KEY"] = "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"
6 # 查询 Tavily 搜索 API
7 search = TavilySearchResults(max_results=1)
8 # 执行查询
9 res = search.invoke("今天上海天气怎么样")
10 print(res)
```

```
1 [{"url": 'http://sh.cma.gov.cn/sh/tqyb/jrtq/', 'content': '上海今天天气温度30°C ~ 38°C，偏南风风力4-5级，有多云和雷阵雨的可能。生活气象指数显示，气温高，人体感觉不舒适，不适宜户外活动。'}]
```

## 7.2.2 定义Retriever

Retriever 是 langchain 库中的一个模块，用于检索工具。

根据上述查询结果中的某个URL中，获取一些数据创建一个检索器。

```
1 from langchain_community.document_loaders import WebBaseLoader
2 from langchain_community.vectorstores import FAISS
3 from langchain_openai import OpenAIEmbeddings
4 from langchain_text_splitters import RecursiveCharacterTextSplitter
5 import os
6 import dotenv
7 dotenv.load_dotenv()
8
9 # 1. 提供一个大模型
10 os.environ["OPENAI_API_KEY"] = os.getenv("OPENAI_API_KEY1")
11 os.environ["OPENAI_BASE_URL"] = os.getenv("OPENAI_BASE_URL")
12
13 embedding_model = OpenAIEmbeddings()
14
15 # 2. 加载HTML内容为一个文档对象
16 loader = WebBaseLoader("https://zh.wikipedia.org/wiki/%E7%8C%AB")
17 docs = loader.load()
18 # print(docs)
19
20 # 3. 分割文档
21 splitter = RecursiveCharacterTextSplitter(
22 chunk_size=1000,
23 chunk_overlap=200
24)
25
26 documents = splitter.split_documents(docs)
27
28 # 4. 向量化 得到向量数据库对象
29 vector = FAISS.from_documents(documents, embedding_model)
30
31 # 5. 创建检索器
```

```

32 retriever = vector.as_retriever()
33
34 # 测试检索结果
35 #print(retriever.invoke("猫的特征")[0])

```

```

1 page_content='聽覺[编辑]
2 貓每隻耳各有32條獨立的肌肉控制耳殼轉動，因此雙耳可單獨朝向不同的音源轉動，使其向獵物移動時
 仍能對周遭其他音源保持直接接觸。[50] 除了蘇格蘭折耳貓這類基因突變的貓以外，貓極少有狗常見的
 「垂耳」，多數的貓耳向上直立。當貓忿怒或受驚時，耳朵會貼向後方，並發出咆哮與「嘶」聲。
3 貓與人類對低頻聲音靈敏度相若。人類中只有極少數的調音師能聽到20 kHz以上的高頻聲音（8.4度的
 八度音），貓則可達64kHz（10度的八度音），比人類要高1.6個八度音，甚至比狗要高1個八度；但是
 貓辨別音差須開最少5度，比起人類辨別音差須開最少0.5度來得粗疏。[51][47]
4
5 嗅覺[编辑]
6 家貓的嗅覺較人類靈敏14倍。[52]貓的鼻腔內有2億個嗅覺受器，數量甚至超過某些品種的狗（狗嗅覺細
 胞約1.25億~2.2億）。
7
8 味覺[编辑]
9 貓早期演化時由於基因突變，失去了甜的味覺，[53]但貓不光能感知酸、苦、鹹味，选择适合自己口味
 的食物，还能尝出水的味道，这一点是其他动物所不及的。不过总括来说猫的味觉不算完善，相比一般
 人類平均有9000個味蕾，貓一般平均僅有473個味蕾且不喜好低於室溫之食物。故此，貓辨認食物乃憑
 嗅覺多於味覺。[47]
10
11 觸覺[编辑]
12 貓在磨蹭時身上會散發出特別的費洛蒙，當這些獨有的費洛蒙留下時，目的就是在宣誓主權，提醒其它
 貓這是我的，其實這種行為算是一種標記地盤的象徵，會讓牠們有感到安心及安全感。
13
14 被毛[编辑]
15 主条目：貓的毛色遺傳和顏色
16 長度[编辑]
17 貓主要可以依據被毛長度分為長毛貓，短毛貓和無毛貓。' metadata={'source':
 'https://zh.wikipedia.org/wiki/%E7%8C%AB', 'title': '猫 - 维基百科，自由的百科全书', 'language':
 'zh'}
18

```

### 7.2.3 创建工具、工具集

前面，我们已经填充了我们将要进行Retriever的索引，我们可以轻松地将其转换为一个工具。

```

1 from langchain.tools.retriever import create_retriever_tool
2
3 # 创建一个工具来检索文档
4 retriever_tool = create_retriever_tool(
5 retriever=retriever,
6 name="wiki_search",
7 description="搜索维基百科",
8)

```

创建工具集：

```
1 tools = [search, retriever_tool]
```

## 7.2.4 语言模型调用工具

```
1 from langchain_openai import ChatOpenAI
2 from langchain_core.messages import HumanMessage
3
4 # 获取大模型
5 model = ChatOpenAI(model="gpt-4o-mini")
6
7 # 模型绑定工具
8 model_with_tools = model.bind_tools(tools)
9
10 # 根据输入自动调用工具
11 messages = [HumanMessage(content="今天上海天气怎么样")]
12 response = model_with_tools.invoke(messages)
13 print(f"ContentString: {response.content}")
14 print(f"ToolCalls: {response.tool_calls}")
15
```

ContentString:

ToolCalls: [{'name': 'tavily\_search\_results\_json', 'args': {'query': '今天上海天气'}, 'id': 'call\_EOxYscVIVjttlbtWoR1CvTm', 'type': 'tool\_call'}]

我们可以看到现在没有内容，但有一个工具调用！它要求我们调用Tavily Search工具。

这并不是在调用该工具，只是告诉我们要调用。为了实际调用它，我们将创建我们的Agent程序。

## 7.2.5 创建Agent程序(使用通用方式)

以FUNCTION CALL方式进行调用：

```
1 from langchain import hub
2 prompt = hub.pull("hwchase17/openai-functions-agent")
3
4 print(prompt.messages)
```

```
[SystemMessagePromptTemplate(prompt=PromptTemplate(input_variables=[],
template='You are a helpful assistant')),
MessagesPlaceholder(variable_name='chat_history', optional=True),
HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['input'],
template='{input}')), MessagesPlaceholder(variable_name='agent_scratchpad')]
```

现在，我们可以使用LLM、提示和工具初始化Agent。Agent 负责接收输入并决定采取什么行动。关键的是，Agent 不执行这些操作 - 这是由AgentExecutor（下一步）完成的。

```

1 from langchain.agents import create_tool_calling_agent
2 from langchain.agents import AgentExecutor
3 # 创建Agent对象
4 agent = create_tool_calling_agent(model, tools, prompt)
5
6 # 创建AgentExecutor对象
7 agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

```

## 7.2.6 运行Agent

现在，我们可以在几个查询上运行Agent！请注意，目前这些都是**无状态**查询（它不会记住先前的交互）。

首先，让我们看看当不需要调用工具时它如何回应：

```

1 print(agent_executor.invoke({"input": "猫的特征"}))

```

```
{'input': '猫的特征', 'output': '猫是一种灵活且适应性强的哺乳动物，具有一系列独特的生理特征和感官能力。以下是猫的主要特征：\n\n### 感官\n1. 听觉：猫的耳朵内有32条独立肌肉，能够高度灵活地朝向不同音源，捕捉声音时非常敏锐。\n2. 视力：猫的夜视能力远超人类，是人类的六倍，猫眼具有特殊的反射膜，可以在微光下观察物体。猫的视觉范围广，但在光线强烈时视野会缩小。\n3. 嗅觉：猫的嗅觉是人类的14倍，鼻腔内有2亿个嗅觉受器，有助于捕猎。\n4. 味觉：由于基因突变，猫失去了甜味的感知，但能够分辨酸、苦和咸等味道。\n5. 触觉：猫身上的触须能感知空气的微小变化，帮助它们在黑暗中导航。'\n\n### 体型特征\n- 爪子：猫的爪子尖锐，可以伸缩，脚掌底部有脂肪垫，可以悄无声息地行走。'\n- 灵活性：猫的舞动非常优雅，能够迅速反应，适应多种环境。'\n\n### 习性\n- 领地意识：猫喜欢通过磨蹭和气味标记来控制领地。'\n- 社交性：虽然猫通常被认为是独立动物，但它们也能够与人类和其他动物建立深厚的关系。'\n\n### 品种\n- 公认\n的猫品种多样，国际猫协会、爱猫协会和国际猫联合会等组织识别了多种标准化品种。'\n\n猫作为一种家庭宠物，能够与人类建立紧密的联系，同时保持其独立性和捕猎本能。它们的特征使它们在各种环境中都能生存和繁衍。'}

```

现在让我们尝试一个需要调用搜索工具的示例：

```

1 print(agent_executor.invoke({"input": "今天上海天气怎么样"}))

```

```
{'input': '今天上海天气怎么样', 'output': '今天上海的天气情况如下：\n\n当前温度：22.9℃\n相对湿度：65%\n体感温度：25.4℃\n降水量：0mm\n空气质量：优\n舒适度：温暖，较舒适\n风速：约7.9m/s，主要风向为东南风\n\n预计在今天气温将最高达到31.5℃。'\n\n如果你想获取更详细的天气信息，可以查看中国气象局-天气预报 或者 中央气象台。'}

```

## 7.2.7 添加记忆

```

1 from langchain_community.chat_message_histories import ChatMessageHistory
2
3 from langchain_core.chat_history import BaseChatMessageHistory
4
5 from langchain_core.runnables.history import RunnableWithMessageHistory
6
7 store = {}

```

```

8
9 # 调取指定session_id对应的memory
10 def get_session_history(session_id: str) -> BaseChatMessageHistory:
11
12 if session_id not in store:
13 store[session_id] = ChatMessageHistory()
14
15 return store[session_id]
16
17 agent_with_chat_history = RunnableWithMessageHistory(
18 runnable=agent_executor,
19 get_session_history=get_session_history,
20 input_messages_key="input",
21 history_messages_key="chat_history",
22)
23
24 response = agent_with_chat_history.invoke(
25 {"input": "Hi, 我的名字是Cyber"},
26 config={"configurable": {"session_id": "123"}},
27)
28
29 print(response)

```

```
{'input': 'Hi, 我的名字是Cyber', 'chat_history': [], 'output': '你好, Cyber! 有什么我可以帮助你的吗? '}
```

继续提问

```

1 response = agent_with_chat_history.invoke(
2 {"input": "我叫什么名字?"},
3 config={"configurable": {"session_id": "123"}},
4)
5
6 print(response)

```

```
{'input': '我叫什么名字?', 'chat_history': [HumanMessage(content='Hi, 我的名字是Cyber', additional_kwargs={}, response_metadata={}), AIMessage(content='你好, Cyber! 有什么我可以帮助你的吗? ', additional_kwargs={}, response_metadata={}), HumanMessage(content='我叫什么名字?', additional_kwargs={}, response_metadata={}), AIMessage(content='你的名字是 Cyber。请问还有其他问题吗? ', additional_kwargs={}, response_metadata={})], 'output': '你的名字是 Cyber。请问你还有其他问题吗? '}
```

继续:

```

1 response = agent_with_chat_history.invoke(
2 {"input": "我叫什么名字?"},
3 config={"configurable": {"session_id": "4566"}},
4)
5 print(response)

```

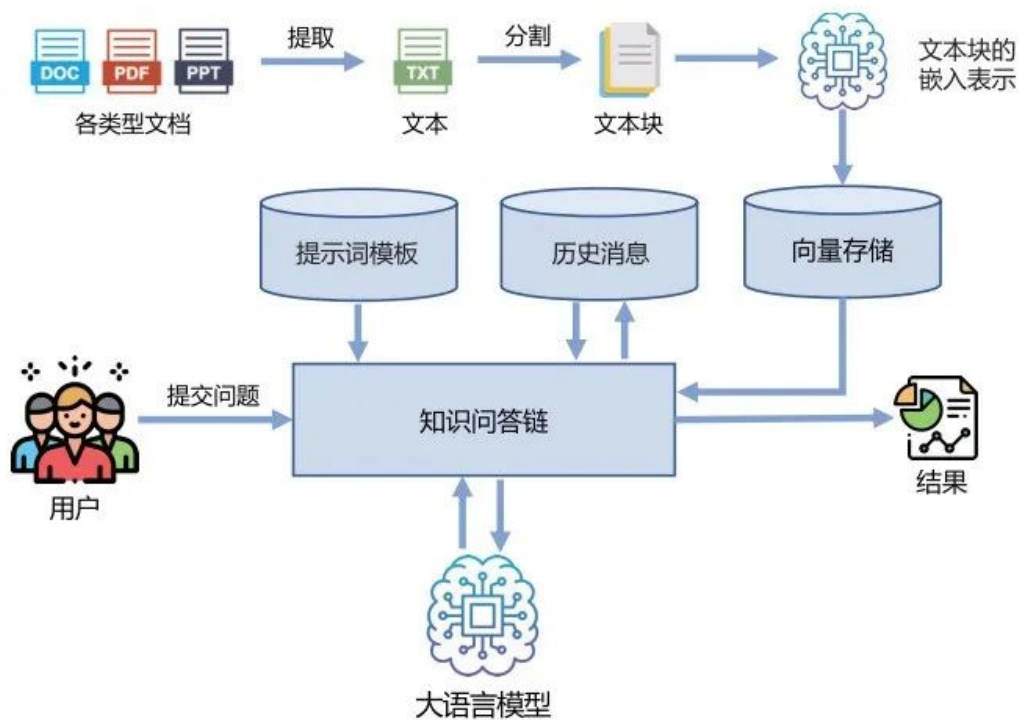


```
{'input': '我叫什么名字?', 'chat_history': [], 'output': '我无法知道你的名字，因为你并没有告诉我。如果你愿意，可以告诉我你的名字，我很乐意记住它并与之交流！'}
```

## 8、项目：知识库问答助手(了解)

大语言模型虽然可以很好地回答很多领域的各种问题，但是由于其知识是通过语言模型训练及指令微调等方式注入模型参数中的，因此针对本地知识库中的内容，大语言模型很难通过此前的方式有效地进行学习。通过 LangChain 框架，可以有效地融合本地知识库内容与大语言模型的知识问答能力。

基于 LangChain 的知识库问答助手框架如下图所示。



知识库问答助手的工作流程主要包含以下几个步骤。

- (1) 收集领域知识数据构造知识库，这些数据应当能够尽可能地全面覆盖问答需求。
- (2) 对知识库中的非结构数据进行文本提取和文本分割，得到文本块。
- (3) 利用嵌入向量表示模型给出文本块的嵌入表示，并利用[向量数据库](#)进行保存。
- (4) 根据用户输入信息的嵌入表示，通过[向量数据库](#)检索得到最相关的文本片段，将提示词模板与用户提交问题及历史消息合并输入大语言模型。
- (5) 将大语言模型结果返回给用户。

安装必要的包

- 1 `pip install docx2txt`
- 2 `pip install langchain-chroma`

- 1 *#导入必须的包*
- 2 *# Import the required packages*
- 3 `from langchain_community.document_loaders import UnstructuredExcelLoader, Docx2txtLoader, PyPDFLoader`

```

4 from langchain_text_splitters import CharacterTextSplitter
5 from langchain_openai import OpenAIEmbeddings
6 from langchain_chroma import Chroma
7 #导入聊天所需的模块
8 # Import the required packages for chat
9 from langchain_openai import ChatOpenAI
10 from langchain_core.prompts import ChatPromptTemplate
11
12
13 #定义chatdoc
14 # Define the ChatDoc class
15 class ChatDoc():
16
17 def __init__(self):
18 self.doc = None
19 self.splitText = [] #分割后的文本 split text
20 self.template = [
21 ("system",
22 "你是一个处理文档的秘书,你从不说自己是一个大模型或者AI助手,你会根据下面提供的上下文
23 内容来继续回答问题.\n 上下文内容\n {context} \n"),
24 ("human", "你好! "),
25 ("ai", "您好, 我是尚硅谷秘书",
26 ("human", "{question}"),
27]
28 self.prompt = ChatPromptTemplate.from_messages(self.template)
29
30 def getFile(self):
31 doc = self.doc
32 loaders = {
33 "docx": Docx2txtLoader
34 }
35 file_extension = doc.split(".")[-1]
36 loader_class = loaders.get(file_extension)
37 if loader_class:
38 try:
39 loader = loader_class(doc)
40 text = loader.load()
41 return text
42 except Exception as e:
43 print(f"Error loading {file_extension} files:{e}")
44 else:
45 print(f"Unsupported file extension: {file_extension}")
46 return None
47
48 #处理文档的函数
49 def splitSentences(self):
50 full_text = self.getFile() #获取文档内容 get the content of the document
51 if full_text != None:
52 #对文档进行分割
53 # Split the document
54 text_split = CharacterTextSplitter(
55 chunk_size=500,
56 chunk_overlap=100,
57 separator="\n\n",
58 length_function=len,

```

```

58 is_separator_regex=False
59)
60 texts = text_split.split_documents(full_text)
61 self.splitText = texts
62
63 #量化与向量存储
64 def embeddingAndVectorDB(self):
65 # embeddings = OpenAIEmbeddings(
66 # model="BAAI/bge-m3",
67 # api_key=os.getenv("SILICON_API_KEY"),
68 # base_url="https://api.siliconflow.cn/v1"
69 #)
70 embeddings = OpenAIEmbeddings()
71
72 db = Chroma.from_documents(
73 documents=self.splitText,
74 embedding=embeddings,
75)
76
77 return db
78
79 #提问并找到相关的文本块
80 def askAndFindFiles(self, question):
81 db = self.embeddingAndVectorDB()
82 print(db._collection.count())
83 #retriever = db.as_retriever(search_type="mmr")
84 retriever = db.as_retriever()
85 return retriever.invoke(input=question)
86
87 #用自然语言和文档聊天
88 def chatWithDoc(self, question):
89 _content = ""
90 context = self.askAndFindFiles(question)
91 for i in context:
92 _content += i.page_content
93 print(f"_{content}", "_content")
94 messages = self.prompt.format_messages(context=_content, question=question)
95 print("message:", messages)
96 llm = ChatOpenAI(model="gpt-4",
97 temperature=0,
98 api_key=os.getenv("OPENAI_API_KEY1"),
99 base_url=os.getenv("OPENAI_BASE_URL"))
100 return llm.invoke(messages)
101
102
103 chat_doc = ChatDoc()
104 chat_doc.doc = "/asset/load/13-sgg_chat.docx"
105 chat_doc.splitSentences()
106 response = chat_doc.chatWithDoc("尚硅谷地址在哪")
107 print(response.content)

```

```
AIMessage(content='尚硅谷教育科技有限公司的总部地址是：北京市海淀区中关村软件园创新大厦B座5层。', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 31, 'prompt_tokens': 1348, 'total_tokens': 1379, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_name': 'gpt-4o-2024-11-20', 'system_fingerprint': 'fp_ee1d74bde0', 'id': 'chatcmpl-BwsYEyERSjGxMNuHenIFJ6AykVs8e', 'service_tier': None, 'finish_reason': 'stop', 'logprobs': None}, id='run--fef4581a-268f-440d-84a4-4cdbbafdfd30-0', usage_metadata={'input_tokens': 1348, 'output_tokens': 31, 'total_tokens': 1379, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}})
```